

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 12_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

You are given a graph, and your task is to find the minimum vertex cover for the given graph. A vertex cover is a subset of vertices in the graph such that every edge in the graph is incident to at least one vertex in the cover.

Input Format

The first line contains an integer 'vertices' representing the number of vertices in the graph.

The second line contains an integer 'edges' representing the number of edges in the graph.

The following 'edges' lines each contain two integers 'u' and 'v' representing an edge between vertices 'u' and 'v'.

Output Format

The output prints the minimum vertex cover for the given graph, where each vertex in the cover is separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7

6

0 1

0 2

1 3

3 4

4 5

5 6

Output: 0 1 3 4 5 6

Answer

```
// You are using #include <stdio.h>
```

```
int main() {
    int V, E;
    scanf("%d", &V);
    scanf("%d", &E);

    int u[505], v[505];
    int visited[105] = {0};

    // read edges
    for (int i = 0; i < E; i++) {
        scanf("%d %d", &u[i], &v[i]);
    }

    // greedy vertex cover
    for (int i = 0; i < E; i++) {
        if (!visited[u[i]] && !visited[v[i]]) {
            visited[u[i]] = 1;
            visited[v[i]] = 1;
        }
    }
}
```

```

    }
}

// print result
for (int i = 0; i < V; i++) {
    if (visited[i])
        printf("%d ", i);
}

return 0;
}g GCC

```

Status : Wrong

Marks : 0/10

2. Problem Statement

A financial analyst is studying the daily profit and loss values of a company over a period of time. Each day's profit or loss is recorded as an integer in an array, where positive values indicate profit and negative values indicate loss.

The analyst wants to identify a continuous period of days during which the company achieved the maximum total profit. This period must consist of consecutive days only, and at least one day must be selected.

Given an array of integers representing daily profit and loss values, determine the maximum possible sum of any contiguous subarray.

To solve this efficiently, you must use a linear-time algorithm.

Input Format

The first line contains an integer n , representing the number of days.

The second line contains n space-separated integers representing daily profit or loss values.

Output Format

Print a single integer representing the maximum sum of a contiguous subarray.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 9

-2 1 -3 4 -1 2 1 -5 4

Output: 6

Answer

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int arr[100000];
```

```
    for (int i = 0; i < n; i++)
```

```
        scanf("%d", &arr[i]);
```

```
    int current = arr[0];
```

```
    int maxSum = arr[0];
```

```
    for (int i = 1; i < n; i++) {
```

```
        if (current < 0)
```

```
            current = arr[i];
```

```
        else
```

```
            current += arr[i];
```

```
        if (current > maxSum)
```

```
            maxSum = current;
```

```
    }
```

```
    printf("%d", maxSum);
```

```
    return 0;
```

```
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Given an array of integers, determine whether there exists a majority element in the array.

A majority element is defined as an element that appears more than $n/2$ times in the array.

Your task is to implement a function `majorityElement()` that takes the input array and returns the majority element if it exists. If no such element exists, return -1.

Formula

An element is considered a majority element if:

$$\text{frequency}(\text{element}) > n/2$$

where n is the total number of elements in the array.

Input Format

The first line of input contains an integer n , representing the number of elements in the array.

The second line contains n space-separated integers $a_1 a_2 a_3 \dots a_n$ representing the array elements.

Output Format

Print the majority element if it exists. Otherwise, print -1.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 9

3 3 4 2 4 4 2 4 4

Output: 4

Answer

```
// You are using GCC#include <stdio.h>
```

```
int main() {
    int n;
    scanf("%d", &n);

    int arr[1005];

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    // Step 1: Find candidate
    int candidate = arr[0];
    int count = 1;

    for (int i = 1; i < n; i++) {
        if (arr[i] == candidate)
            count++;
        else
            count--;

        if (count == 0) {
            candidate = arr[i];
            count = 1;
        }
    }

    // Step 2: Verify
    count = 0;
    for (int i = 0; i < n; i++) {
        if (arr[i] == candidate)
            count++;
    }

    if (count > n / 2)
        printf("%d", candidate);
    else
        printf("-1");
}
```

```
    return 0;  
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Aarav is analyzing temperature fluctuations recorded over several days. The temperature changes are stored in an array of integers. He wants to identify a contiguous period of days such that the product of the temperature changes during that period is maximized.

Given an array of integers, determine the maximum product that can be obtained from any contiguous subarray containing at least one element.

Your task is to compute this value efficiently.

Formula

To correctly handle both positive and negative values, track both the maximum and minimum product ending at each index.

If the current element is negative, swap the previous maximum and minimum values before updating.

At each index i :

$\text{currentMax} = \max(\text{arr}[i], \text{arr}[i] \times \text{currentMax})$

$\text{currentMin} = \min(\text{arr}[i], \text{arr}[i] \times \text{currentMin})$

The final answer is the maximum value obtained among all currentMax values.

Input Format

The first line contains an integer n , representing the number of days.

The second line contains n space-separated integers representing the temperature change factors.

Output Format

The first line contains an integer n, representing the number of days.

The second line contains n space-separated integers representing the temperature change factors.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

2 3 -2 4 -1 5

Output: 240

Answer

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int arr[1000];
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    // Boyer-Moore Voting
```

```
    int candidate = 0, count = 0;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (count == 0) {
```

```
            candidate = arr[i];
```

```
            count = 1;
```

```
        } else if (arr[i] == candidate) {
```

```
            count++;
```

```
        } else {
```

```
            count--;
```

```
        }
```

```
    }
```



```

// Verify
count = 0;
for (int i = 0; i < n; i++) {
    if (arr[i] == candidate)
        count++;
}

if (count > n / 2)
    printf("%d", candidate);
else
    printf("-1");

return 0;
}

```

Status : Wrong

Marks : 0/10

5. Problem Statement

Sidhant is a logistics manager responsible for planning delivery routes for a fleet of trucks. He has a list of cities, and the distances between every pair of cities are given in the form of a matrix.

Each truck must start from a central warehouse (city 0), visit every other city exactly once, and return back to the warehouse. The goal is to minimize the total travel distance.

Sidhant needs to determine the minimum distance required to complete the route and also print the optimal path taken.

Input Format

The first line contains an integer n , representing the number of cities.

The next n lines contain n space-separated integers, representing the distance matrix.

Output Format

The first line of output prints the minimum travel distance.

In the next line, print the optimal path starting and ending at city 0.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output: 80

0 1 3 2 0

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int n;
```

```
int cost[20][20];
```

```
int visited[20];
```

```
int path[20], bestPath[20];
```

```
int minCost = INT_MAX;
```

```
void tsp(int pos, int count, int currCost) {
```

```
    if (count == n && cost[pos][0] != 0) {  
        int totalCost = currCost + cost[pos][0];
```

```
        if (totalCost < minCost) {  
            minCost = totalCost;
```

```
            for (int i = 0; i < n; i++)  
                bestPath[i] = path[i];
```

```
        }
```

```
        return;
```

```
    }
```

```

    for (int i = 0; i < n; i++) {
        if (!visited[i] && cost[pos][i] != 0) {

            visited[i] = 1;
            path[count] = i;

            tsp(i, count + 1, currCost + cost[pos][i]);

            visited[i] = 0;
        }
    }
}

```

```

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    visited[0] = 1;
    path[0] = 0;

    tsp(0, 1, 0);

    printf("%d\n", minCost);

    for (int i = 0; i < n; i++)
        printf("%d ", bestPath[i]);

    printf("0");

    return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 12_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Meera is preparing for a science exhibition and needs to select a group of items such that their total weight is exactly equal to a given target weight. Each item has a known weight, and every item can be selected at most once.

Given a list of item weights and a target weight k , determine whether there exists a subset of items whose total weight is exactly k .

If such a subset exists, print any one valid subset.

If no such subset exists, print -1.

The solution must be implemented using Dynamic Programming.

Formula

Let

$dp[i][j] = \text{true}$

if a subset of the first i items can form a sum j .

Transition:

If $arr[i-1] > j$

$dp[i][j] = dp[i-1][j]$

Else

$dp[i][j] = dp[i-1][j] \text{ OR } dp[i-1][j - arr[i-1]]$

Input Format

The first line contains two integers n and k , where n is the number of items and k is the target weight.

The second line contains n space-separated integers representing item weights.

Output Format

If a valid subset exists, print the elements of one such subset in a single line separated by spaces.

If no subset exists, print -1.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 9
3 3 4 4 12 5
Output: 5 4

Answer

// You are using GCC#include <stdio.h>

```
int main() {
    int n, k;
    scanf("%d %d", &n, &k);

    int arr[105];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    int dp[105][1005] = {0};

    // base case
    for (int i = 0; i <= n; i++)
        dp[i][0] = 1;

    // fill DP table
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= k; j++) {
            if (arr[i-1] > j)
                dp[i][j] = dp[i-1][j];
            else
                dp[i][j] = dp[i-1][j] || dp[i-1][j - arr[i-1]];
        }
    }

    if (!dp[n][k]) {
        printf("-1");
        return 0;
    }

    // reconstruct subset
    int i = n, j = k;

    while (i > 0 && j > 0) {
        if (dp[i-1][j]) {
            i--;
        }
    }
}
```

```

    } else {
        printf("%d ", arr[i-1]);
        j -= arr[i-1];
        i--;
    }
}

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Ramya is working as a logistics coordinator for a delivery company. Every day, she receives a list of delivery locations arranged along a straight highway. Each location has a certain delivery priority value (which may be positive or negative depending on profit or loss).

Ramya wants to choose a continuous sequence of delivery locations such that the total delivery priority is maximized, ensuring maximum profit for that day.

Since the company handles a large number of deliveries, Ramya needs an efficient solution that runs in polynomial time.

Input Format

The first line contains an integer n , representing the number of delivery locations.

The second line contains n space-separated integers, representing the priority values.

Output Format

The output prints a single integer representing the maximum total priority of any contiguous sequence of locations.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 8

-2 3 5 -1 4 -5 2 6

Output: 14

Answer

```
// You are using GCC#include <stdio.h>
```

```
int main() {
    int n;
    scanf("%d", &n);

    int arr[100000];

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    int current_sum = arr[0];
    int max_sum = arr[0];

    for (int i = 1; i < n; i++) {
        if (current_sum + arr[i] > arr[i])
            current_sum = current_sum + arr[i];
        else
            current_sum = arr[i];

        if (current_sum > max_sum)
            max_sum = current_sum;
    }

    printf("%d", max_sum);

    return 0;
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Ragav is managing a chain of warehouses located along a long highway. Each warehouse is positioned at a specific coordinate on the road.

To improve efficiency, Ragav wants to choose a central hub location such that the total travel distance from all warehouses to this hub is minimized.

He can place the hub at any one of the existing warehouse locations.

Since the number of warehouses can be very large, Ragav needs an efficient algorithm that runs in polynomial time to compute the optimal location.

Input Format

The first line contains an integer n , representing the number of warehouses.

The second line contains n space-separated integers, representing the positions of warehouses on the highway.

Output Format

The output prints a single integer representing the minimum total distance from all warehouses to the chosen hub.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

1 2 3 6 10

Output: 13

Answer

```
// You are using GCC#include <stdio.h>
#include <stdlib.h>
```

```
int compare(const void *a, const void *b) {
    long long x = *(long long*)a;
    long long y = *(long long*)b;
    return (x > y) - (x < y);
}
```

```

}

int main() {
    int n;
    scanf("%d", &n);

    long long arr[100000];

    for (int i = 0; i < n; i++)
        scanf("%lld", &arr[i]);

    // sort array
    qsort(arr, n, sizeof(long long), compare);

    // find median
    long long median = arr[n / 2];

    long long total = 0;

    // calculate total distance
    for (int i = 0; i < n; i++) {
        if (arr[i] > median)
            total += arr[i] - median;
        else
            total += median - arr[i];
    }

    printf("%lld", total);

    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Peter is a project manager in a software company. He has a list of employees, and each employee has a certain skill contribution value (which can be positive or negative depending on their impact on the project).

Peter wants to form a team such that the total skill contribution is exactly equal to a given target value. Each employee can be selected at most once, and the team must contain at least one employee.

Since the number of possible teams grows rapidly with the number of employees, Peter needs to determine whether such a team exists.

Input Format

The first line contains an integer n , representing the number of employees.

The second line contains n space-separated integers, representing the skill contribution of each employee.

The third line contains an integer $target$, representing the required total skill.

Output Format

If a valid team exists, the output prints: "Team Found: x_1 x_2 x_3 ..."

If no such team exists, the output prints: "No valid team found"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
3 7 -2 5 1
6

Output: Team Found: 3 -2 5

Answer

```
// You are using GCC#include <stdio.h>
```

```
int n, target;  
int arr[25];  
int subset[25];  
int found = 0;
```

```
void solve(int index, int count, int sum) {
```

```

    if (sum == target && count > 0) {
        printf("Team Found: ");
        for (int i = 0; i < count; i++) {
            printf("%d ", subset[i]);
        }
        found = 1;
        return;
    }

    if (index == n || found)
        return;

    // include
    subset[count] = arr[index];
    solve(index + 1, count + 1, sum + arr[index]);

    // exclude
    solve(index + 1, count, sum);
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    scanf("%d", &target);

    solve(0, 0, 0);

    if (!found) {
        printf("No valid team found");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Viyana is an event coordinator organizing multiple sessions for a tech conference. Each session has a start time and an end time.

Due to limited resources, Viyana wants to select a group of sessions such that:

No two selected sessions overlap in time. The total number of selected sessions is maximized.

However, to make the event more impactful, each session also has a priority score, and Viyana wants to maximize the total priority score instead of just the count.

Your task is to help Viyana select the optimal set of non-overlapping sessions that yields the maximum total priority score.

Input Format

The first line contains an integer n , representing the number of sessions.

The next n lines each contain three integers: start time, end time and priority score.

Output Format

The first line of output prints the maximum total priority score.

In the next line, print the selected sessions (indices starting from 1) in ascending order.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

1 3 5

2 5 6

4 6 5

6 7 4

Output: 14

1 3 4

Answer

// You are using GCC#include <stdio.h>

int n;

int start[25], end[25], profit[25];

int bestProfit = 0;

int bestSet[25], bestSize = 0;

int currentSet[25];

int isValid(int idx, int size) {

for (int i = 0; i < size; i++) {

int j = currentSet[i];

if (!(end[j] <= start[idx] || end[idx] <= start[j]))

return 0; // overlap

}

return 1;

}

void solve(int index, int size, int total) {

if (index == n) {

if (total > bestProfit) {

bestProfit = total;

bestSize = size;

for (int i = 0; i < size; i++)

bestSet[i] = currentSet[i];

}

return;

}

// include

if (isValid(index, size)) {

currentSet[size] = index;

solve(index + 1, size + 1, total + profit[index]);

```
}  
    // exclude  
    solve(index + 1, size, total);  
}  
  
int main() {  
    scanf("%d", &n);  
  
    for (int i = 0; i < n; i++) {  
        scanf("%d %d %d", &start[i], &end[i], &profit[i]);  
    }  
  
    solve(0, 0, 0);  
  
    printf("%d\n", bestProfit);  
  
    // print indices (1-based)  
    for (int i = 0; i < bestSize; i++) {  
        printf("%d ", bestSet[i] + 1);  
    }  
  
    return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 12_COD

Attempt : 1

Total Mark : 30

Marks Obtained : 29

Section 1 : Coding

1. Problem Statement

Raj is developing financial software that must identify all non-empty subsets of a given list of expenses whose total sum equals a specified target value.

Each expense can either be included or excluded from a subset. The same expense cannot be used more than once in a subset.

Your task is to print all such valid subsets. If no valid non-empty subset exists whose sum equals the target value, print:

No subset matches the target sum

This problem demonstrates how the number of possible subsets increases exponentially when using brute-force approaches.

Formula

Subset Sum = sum of selected elements in the subset

A subset is considered valid if:

$\text{sum}(\text{subset elements}) = \text{target}$

Input Format

The first line contains an integer n , representing the number of expenses.

The second line contains n space-separated integers representing the expense values.

The third line contains an integer t , representing the target sum.

Output Format

Print each valid non-empty subset whose elements sum to t , one subset per line.

Elements of a subset must be printed as space-separated integers.

If no such subset exists, print:

No subset matches the target sum

Refer to the sample output for formatting.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

1 2 3

3

Output: 1 2

3

Answer

```
// You are using GCC#include <stdio.h>
```

```
int n, target;  
int arr[25];  
int subset[25];
```

```
int found = 0;
```

```
void solve(int index, int count, int sum) {  
    // if sum matches and subset not empty  
    if (sum == target && count > 0) {  
        for (int i = 0; i < count; i++) {  
            printf("%d ", subset[i]);  
        }  
        printf("\n");  
        found = 1;  
        return;  
    }
```

```
    if (index == n)  
        return;
```

```
    // include element  
    subset[count] = arr[index];  
    solve(index + 1, count + 1, sum + arr[index]);
```

```
    // exclude element  
    solve(index + 1, count, sum);  
}
```

```
int main() {  
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++)  
        scanf("%d", &arr[i]);
```

```
    scanf("%d", &target);
```

```
    solve(0, 0, 0);
```

```
    if (!found) {
```

```
        printf("No subset matches the target sum");  
    }  
  
    return 0;  
}
```

Status : Partially correct

Marks : 9/10

2. Problem Statement

Sanjana is a cybersecurity analyst responsible for securing communication in a corporate network. The network consists of multiple computers (nodes) connected by communication links (edges).

To ensure secure communication, Sanjana wants to select a group of computers such that every pair of selected computers is directly connected to each other. This ensures a fully trusted group where all members can communicate securely without intermediaries. Such a group is called a clique.

Sanjana is given a list of connections in the network, and she needs to determine whether there exists a clique of size k .

Input Format

The first line contains two integers n and m , representing the number of computers (nodes) and connections (edges).

The next m lines each contain two integers u and v , representing a connection between nodes u and v .

The last line contains an integer k , representing the required clique size.

Output Format

If a clique of size k exists, the output prints: "Clique Found: $v_1 v_2 v_3 \dots$ "

Otherwise, the output prints: "No clique of given size exists"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 6

0 1

0 2

1 2

1 3

2 3

3 4

3

Output: Clique Found: 0 1 2

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int n, m, k;
```

```
int adj[25][25];
```

```
int clique[25];
```

```
int found = 0;
```

```
// check if vertex can be added
```

```
int isSafe(int v, int size) {
```

```
    for (int i = 0; i < size; i++) {
```

```
        if (!adj[v][clique[i]])
```

```
            return 0;
```

```
    }
```

```
    return 1;
```

```
}
```

```
void solve(int start, int size) {
```

```
    if (size == k) {
```

```
        printf("Clique Found: ");
```

```
        for (int i = 0; i < k; i++) {
```

```
            printf("%d ", clique[i]);
```

```
        }
```

```
        found = 1;
```

```
        return;
```

```
    }
```

```

    for (int i = start; i < n; i++) {
        if (isSafe(i, size)) {
            clique[size] = i;
            solve(i + 1, size + 1);

            if (found) return; // stop after first clique
        }
    }
}

```

```

int main() {
    scanf("%d %d", &n, &m);

    // initialize
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            adj[i][j] = 0;

    int u, v;

    // input edges
    for (int i = 0; i < m; i++) {
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
        adj[v][u] = 1;
    }

    scanf("%d", &k);

    solve(0, 0);

    if (!found) {
        printf("No clique of given size exists");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Rehan, a logistics manager, is packing valuable items into two separate containers. Each item has a specific value, and he wants both containers to carry the same total value to ensure balance and fairness. He needs to decide if it's possible to divide the items this way and print one such valid division if it exists.

This problem is based on a classic NP-complete problem known as the Partition Problem, where finding a solution may require checking all combinations in the worst case.

Help him write a program to solve this problem.

Input Format

The first line of input consists of an integer n , representing the number of items.

The second line contains n space-separated integers, where each integer represents the value of an item.

Output Format

If a valid equal partition exists, print:

- Subset 1: $x_1 x_2 x_3 \dots$
- Subset 2: $y_1 y_2 y_3 \dots$

If no such partition exists, it prints "No valid equal partition exists."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

1 5 11 5

Output: Subset 1: 1 5 5

Subset 2: 11

Answer

```
// You are using GCC#include <stdio.h>
```

```

int n;
int arr[20];
int subset[20];
int used[20];
int target;
int found = 0;

void solve(int index, int count, int sum) {
    if (sum == target) {
        // print subset 1
        printf("Subset 1: ");
        for (int i = 0; i < count; i++)
            printf("%d ", subset[i]);
        printf("\n");

        // mark used elements
        for (int i = 0; i < count; i++) {
            for (int j = 0; j < n; j++) {
                if (!used[j] && arr[j] == subset[i]) {
                    used[j] = 1;
                    break;
                }
            }
        }

        // print subset 2
        printf("Subset 2: ");
        for (int i = 0; i < n; i++) {
            if (!used[i])
                printf("%d ", arr[i]);
        }

        found = 1;
        return;
    }

    if (index == n || sum > target)
        return;

    //include
    subset[count] = arr[index];
    solve(index + 1, count + 1, sum + arr[index]);
}

```

```

    if (found) return;

    // exclude
    solve(index + 1, count, sum);
}

int main() {
    scanf("%d", &n);

    int total = 0;

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        used[i] = 0;
        total += arr[i];
    }

    if (total % 2 != 0) {
        printf("No valid equal partition exists.");
        return 0;
    }

    target = total / 2;

    solve(0, 0, 0);

    if (!found) {
        printf("No valid equal partition exists.");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 12_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 14

Section 1 : MCQ

1. How many conditions have to be met if an NP-complete problem is polynomially reducible?

Answer

2

Status : Correct

Marks : 1/1

2. _____ is the class of decision problems that can be solved by non-deterministic polynomial algorithms.

Answer

NP

Status : Correct

Marks : 1/1

3. To which of the following classes does a CNF-satisfiability problem belong?

Answer

NP complete

Status : Correct

Marks : 1/1

4. For problems X and Y, Y is NP-complete and X reduces to Y in polynomial time. Which of the following is TRUE?

Answer

X is in NP, but not necessarily NP-complete

Status : Correct

Marks : 1/1

5. A problem in NP is NP-complete if

Answer

The 3-SAT problem can be reduced to it in polynomial time

Status : Correct

Marks : 1/1

6. Which class contains problems that can be solved in polynomial time using a deterministic machine?

Answer

P

Status : Correct

Marks : 1/1

7. What is the key property of problems in NP?

Answer

Their solutions can be verified in polynomial time

Status : Correct

Marks : 1/1

8. What does it mean if a problem is in both NP and NP-Complete?

Answer

It is the hardest among NP problems

Status : Correct

Marks : 1/1

9. If $P = NP$ is ever proven, what would it imply?

Answer

All problems in NP can be solved in polynomial time

Status : Correct

Marks : 1/1

10. What is the class NP named after?

Answer

Nondeterministic Polynomial time

Status : Correct

Marks : 1/1

11. Which complexity class includes problems that are at least as hard as the hardest problems in NP?

Answer

NP-hard

Status : Correct

Marks : 1/1

12. What is the defining property of NP-hard problems?

Answer

They are at least as hard as the hardest problems in NP.

Status : Correct

Marks : 1/1

13. In computational complexity theory, what does "NP" stand for?

Answer

Non-Deterministic Polynomial Time

Status : Correct

Marks : 1/1

14. What is the main characteristic that distinguishes NP-complete problems from NP-hard problems?

Answer

NP-complete problems are the hardest problems in NP.

Status : Correct

Marks : 1/1

15. The concept of "polynomial-time reduction" is commonly used to establish which of the following relationships between problems?

Answer

Status : Skipped

Marks : 0/1

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 11_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement:

John is developing a recommendation system for a social networking platform. The system suggests potential friends to users based on their interests and mutual connections. To enhance the user experience, you want to ensure that the recommendation algorithm considers the removal of certain users from the network due to various reasons such as account deletion or user activity.

However, he still wants to maintain effective friend suggestions even after removing these users. John plan to integrate a functionality that calculates the maximum matching in the bipartite graph representing the user network after removing specific users and their connections.

Help him to achieve the task.

Input Format

Enter the number of vertices in partition

Enter the number of vertices in partition

Enter the adjacency matrix representation of the bipartite graph

Enter the vertex to remove

Output Format

Maximum matching after removing vertex

Sample Test Case

Input: 3

2

0 1

1 0

0 0

1

Output: Maximum matching after removing vertex 1: 1

Answer

```
// You are using GCC#include <stdio.h>
```

```
int n, m;  
int adj[10][10];  
int matchR[10];  
int visited[10];  
int removeV;
```

```
int dfs(int u) {  
    for (int v = 0; v < m; v++) {  
        if (adj[u][v] && !visited[v]) {  
            visited[v] = 1;  
  
            if (matchR[v] == -1 || dfs(matchR[v])) {  
                matchR[v] = u;  
                return 1;  
            }  
        }  
    }  
}
```

```

    return 0;
}

int main() {
    scanf("%d", &n);
    scanf("%d", &m);

    // read adjacency matrix
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &adj[i][j]);

    scanf("%d", &removeV);

    // initialize
    for (int i = 0; i < m; i++)
        matchR[i] = -1;

    int result = 0;

    // run matching excluding removed vertex
    for (int i = 0; i < n; i++) {

        if (i == removeV) continue; // ? skip removed vertex

        for (int j = 0; j < m; j++)
            visited[j] = 0;

        if (dfs(i))
            result++;
    }

    printf("Maximum matching after removing vertex %d: %d", removeV, result);

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Charlie is working on optimizing the hiring process for interns at a tech startup. There are 4 open positions and 4 applicants, each having different skill sets.

Charlie wants to design a system that matches applicants to positions based on their suitability. The suitability is represented using a binary matrix, where:

1 indicates the applicant is suitable for the position. 0 indicates the applicant is not suitable for the position.

Each applicant can be assigned to only one position, and each position can accept only one applicant.

The goal is to determine the maximum number of positions that can be filled by assigning suitable applicants.

Input Format

The input consists of 4 lines.

Each line contains 4 space-separated integers (0 or 1).

Each row represents an applicant.

Each column represents a job position.

Output Format

The output prints the result in the following format: "The job can hire up to X applicants" where X represents the maximum number of applicants that can be assigned to positions.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1 1 0 0

1 0 0 0

0 0 1 1

0 0 0 1

Output: The job can hire up to 4 applicants

Answer

```
// You are using GCC#include <stdio.h>
```

```
int adj[4][4];
int match[4];
int visited[4];
```

```
int dfs(int u) {
    for (int v = 0; v < 4; v++) {
        if (adj[u][v] && !visited[v]) {
            visited[v] = 1;

            if (match[v] == -1 || dfs(match[v])) {
                match[v] = u;
                return 1;
            }
        }
    }
    return 0;
}
```

```
int main() {

    // read 4x4 matrix
    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 4; j++)
            scanf("%d", &adj[i][j]);
```

```
    // initialize
    for (int i = 0; i < 4; i++)
        match[i] = -1;
```

```
    int result = 0;
```

```
    // maximum matching
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++)
            visited[j] = 0;
```

```
        if (dfs(i))
```

```
        result++;
    }

    printf("The job can hire up to %d applicants", result);

    return 0;
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Emily is working on a project that involves managing relationships between two sets of entities, set U and set V. She wants to determine the maximum matching in a bipartite graph to optimize connections without overlap.

Emily also needs the ability to remove a specific connection (edge) between two entities in order to analyze different scenarios and measure the impact on the size of the maximum matching.

Your task is to compute the size of the maximum matching before and after removing a given edge.

Note: Vertices in set U are numbered from 1 to n, and vertices in set V are numbered from 1 to m.

Input Format

The first line contains an integer n, representing the number of vertices in set U.

The second line contains an integer m, representing the number of vertices in set V.

The third line contains an integer e, representing the number of edges.

The next e lines each contain two integers u and v, representing an edge between vertex u (from set U) and vertex v (from set V).

The last line contains two integers u and v, representing the edge that must be

removed.

Output Format

The first line displays: "Size of maximum matching before edge removal: X"

The second line displays: "Size of maximum matching after edge removal: Y"

Refer to the sample output for the formatting specifications

Sample Test Case

Input: 3

3

3

0 1

0 2

1 2

1 2

Output: Size of maximum matching before edge removal: 1

Size of maximum matching after edge removal: 0

Answer

```
// You are using GCC#include <stdio.h>
```

```
int n, m;  
int adj[105][105];  
int matchR[105];  
int visited[105];
```

```
int dfs(int u) {  
    for (int v = 1; v <= m; v++) {  
        if (adj[u][v] && !visited[v]) {  
            visited[v] = 1;  
  
            if (matchR[v] == -1 || dfs(matchR[v])) {  
                matchR[v] = u;  
                return 1;  
            }  
        }  
    }  
}
```

```

        return 0;
    }

    int maxMatching() {
        for (int i = 1; i <= m; i++)
            matchR[i] = -1;

        int result = 0;

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++)
                visited[j] = 0;

            if (dfs(i))
                result++;
        }

        return result;
    }

```

```

int main() {
    int e;
    scanf("%d", &n);
    scanf("%d", &m);
    scanf("%d", &e);

    // initialize
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            adj[i][j] = 0;

    int u, v;

    // input edges
    for (int i = 0; i < e; i++) {
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
    }

    // before removal
    int before = maxMatching();

```

```

// remove edge
scanf("%d %d", &u, &v);
adj[u][v] = 0;

// after removal
int after = maxMatching();

printf("Size of maximum matching before edge removal: %d\n", before);
printf("Size of maximum matching after edge removal: %d", after);

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

In a computer lab, Alex is experimenting with task scheduling on a cluster of processors. The lab has 7 tasks that need to be executed and 5 processors available.

Each task can run only on certain processors due to compatibility constraints. These compatibilities are represented using a binary matrix, where:

1 indicates the task can run on that processor. 0 indicates the task cannot run on that processor.

Each processor can execute only one task at a time, and each task can be assigned to only one processor. Alex wants to determine the maximum number of tasks that can be executed simultaneously.

Write a program to compute the maximum number of compatible task-processor assignments.

Input Format

The input consists of 7 lines.

Each line contains 5 space-separated integers (0 or 1).

Each row represents a task.

Each column represents a processor.

Output Format

The output prints the result in the format: "The processors can run a maximum of x tasks at one time." , where x is the maximum number of tasks that can be executed simultaneously.

Refer to the sample output for formatting specifications.

Sample Test Case

```
Input: 1 1 0 0 0
0 1 0 0 0
0 0 1 1 0
0 0 0 1 1
1 0 0 0 0
0 0 1 0 0
0 0 0 0 1
```

Output: The processors can run a maximum of 5 tasks at one time.

Answer

```
// Yo#include <stdio.h>
```

```
int adj[7][5];
int match[5];
int visited[5];
```

```
int dfs(int u) {
    for (int v = 0; v < 5; v++) {
        if (adj[u][v] && !visited[v]) {
            visited[v] = 1;

            if (match[v] == -1 || dfs(match[v])) {
                match[v] = u;
                return 1;
            }
        }
    }
}
```

```

    return 0;
}

int main() {

    // read 7x5 matrix
    for (int i = 0; i < 7; i++)
        for (int j = 0; j < 5; j++)
            scanf("%d", &adj[i][j]);

    // initialize
    for (int i = 0; i < 5; i++)
        match[i] = -1;

    int result = 0;

    // maximum matching
    for (int i = 0; i < 7; i++) {
        for (int j = 0; j < 5; j++)
            visited[j] = 0;

        if (dfs(i))
            result++;
    }

    printf("The processors can run a maximum of %d tasks at one time.", result);

    return 0;
}

```

u are using Java

Status : Wrong

Marks : 0/10

5. Problem Statement

Alex is working on a project that involves pairing students from two different classes for a team project. The project's success depends on maximizing the number of compatible pairs between these two groups. Each student from one group can be paired with a student from the other group, but no student can belong to more than one pair.

Alex represents the compatibility between students using a binary adjacency matrix, where:

1 indicates that the students are compatible. 0 indicates that the students are not compatible.

Write a program to determine the maximum number of compatible student pairs that can be formed.

Input Format

The first line contains two integers n and m , representing the number of students in the first group and second group, respectively.

The next n lines each contain m space-separated integers, representing the adjacency matrix of student compatibility.

Output Format

The output prints a single line in the following format: "Maximum Bipartite Matching: X", where X is the maximum number of compatible student pairs.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4 3

1 1 0

1 0 1

0 1 0

1 0 0

Output: Maximum Bipartite Matching: 3

Answer

```
#include <stdio.h>
```

```
int n, m;
```

```
int adj[105][105];
```

```
int matchR[105];
```

```
int dfs(int u, int visited[]) {  
    for (int v = 0; v < m; v++) {
```



```

        if (adj[u][v] && !visited[v]) {
            visited[v] = 1;

            if (matchR[v] == -1 || dfs(matchR[v], visited)) {
                matchR[v] = u;
                return 1;
            }
        }
    }
    return 0;
}

```

```

int main() {
    scanf("%d %d", &n, &m);

    // read matrix
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &adj[i][j]);

    // initialize matching
    for (int i = 0; i < m; i++)
        matchR[i] = -1;

    int result = 0;

    // try matching for each node in left set
    for (int i = 0; i < n; i++) {
        int visited[105] = {0}; // ? reset for each DFS

        if (dfs(i, visited))
            result++;
    }

    printf("Maximum Bipartite Matching: %d", result);

    return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 11_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 42.5

Section 1 : Coding

1. Problem Statement

You are given a bipartite graph represented using an edge list. The graph contains two sets of nodes tasks and processors. Each edge connects a task node to a processor node, indicating that the processor can execute that task.

Your goal is to determine the maximum matching in the bipartite graph. A matching is a set of edges such that:

Each task is assigned to at most one processor. Each processor is assigned to at most one task.

Write a program to compute the maximum number of task–processor assignments possible.

Input Format

The first line contains an integer T, representing the number of tasks.

The second line contains an integer P, representing the number of processors.

The third line contains an integer E, representing the number of edges in the bipartite graph.

The next E lines each contain two space-separated integers task and processor where this indicates that the given task can be assigned to the specified processor.

Output Format

The output prints the result in the following format: "The maximum matching is: X", where X is the maximum number of matches that can be made in the bipartite graph.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

3

5

1 1

2 2

4 3

3 3

4 1

Output: The maximum matching is: 3

Answer

```
#include <stdio.h>
```

```
int T, P, E;
```

```
int adj[11][11];
```

```
int matchP[11];
```

```
int visited[11];
```

```
int dfs(int task) {
```

```
    for (int p = 1; p <= P; p++) {
```

```

        if (adj[task][p] && !visited[p]) {
            visited[p] = 1;

            if (matchP[p] == 0 || dfs(matchP[p])) {
                matchP[p] = task;
                return 1;
            }
        }
    }
    return 0;
}

```

```

int main() {
    scanf("%d", &T);
    scanf("%d", &P);
    scanf("%d", &E);

    // initialize
    for (int i = 1; i <= T; i++)
        for (int j = 1; j <= P; j++)
            adj[i][j] = 0;

    for (int i = 1; i <= P; i++)
        matchP[i] = 0;

```

```

    int t, p;

```

```

    // input edges
    for (int i = 0; i < E; i++) {
        scanf("%d %d", &t, &p);
        adj[t][p] = 1;
    }

```

```

    int result = 0;

```

```

    for (int i = 1; i <= T; i++) {
        for (int j = 1; j <= P; j++)
            visited[j] = 0;

```

```

        if (dfs(i))
            result++;
    }

```

```
printf("The maximum matching is: %d", result);  
    return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Irfan is working on a project that involves managing relationships between two sets of entities, set U and set V. He wants to determine the maximum matching in a bipartite graph to optimize connections without overlap. Irfan also wants to explore different scenarios by adding a new connection (edge) between two entities and observing how it affects the maximum matching.

Your task is to compute the size of the maximum matching before and after adding a specified edge.

Note: Vertices in set U are numbered from 0 to $n-1$, and vertices in set V are numbered from 1 to m .

Input Format

The first line contains an integer n , representing the number of vertices in set U.

The second line contains an integer m , representing the number of vertices in set V.

The third line contains an integer e , representing the number of edges.

The next e lines each contain two integers u and v , representing an edge between vertex u from set U and vertex v from set V.

The last line contains two integers u and v , representing the edge to be added.

Output Format

The output prints the results in the following format:

Size of maximum matching before adding edges: X

Size of maximum matching after adding edges: Y

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

4

3

0 1

0 2

1 1

1 2

Output: Size of maximum matching before adding edges: 1

Size of maximum matching after adding edges: 1

Answer

// You are using GCC#include <stdio.h>

```
int n, m;
int adj[105][105];
int matchV[105];
int visited[105];

int dfs(int u) {
    for (int v = 1; v <= m; v++) {
        if (adj[u][v] && !visited[v]) {
            visited[v] = 1;

            if (matchV[v] == -1 || dfs(matchV[v])) {
                matchV[v] = u;
                return 1;
            }
        }
    }
    return 0;
}
```

```
int maxMatching() {  
    for (int i = 1; i <= m; i++)  
        matchV[i] = -1;
```

```
    int result = 0;
```

```
    for (int i = 0; i < n; i++) {  
        for (int j = 1; j <= m; j++)  
            visited[j] = 0;
```

```
        if (dfs(i))  
            result++;
```

```
    }
```

```
    return result;
```

```
}
```

```
int main() {
```

```
    int e;
```

```
    scanf("%d", &n);
```

```
    scanf("%d", &m);
```

```
    scanf("%d", &e);
```

```
    // initialize
```

```
    for (int i = 0; i < n; i++)
```

```
        for (int j = 1; j <= m; j++)
```

```
            adj[i][j] = 0;
```

```
    int u, v;
```

```
    // input edges
```

```
    for (int i = 0; i < e; i++) {
```

```
        scanf("%d %d", &u, &v);
```

```
        adj[u][v] = 1;
```

```
    }
```

```
    // compute before
```

```
    int before = maxMatching();
```

```
    // add new edge
```

```
    scanf("%d %d", &u, &v);
```

```
    adj[u][v] = 1;
```

```

// compute after
int after = maxMatching();

printf("Size of maximum matching before adding edges: %d\n", before);
printf("Size of maximum matching after adding edges: %d", after);

return 0;
}

```

Status : Partially correct

Marks : 2.5/10

3. Problem Statement

Jack wants to create a program to manage a bipartite graph, which contains two sets of vertices. He needs the program to:

Initialize the graph with a specific size, Add edges between the two sets of vertices, and Find pairings where each vertex from the first set is matched with at most one vertex from the second set, and no two vertices from the first set are matched with the same vertex from the second set.

The goal is to determine the maximum matching pairs in the bipartite graph.

Input Format

The first line contains an integer m , representing the number of vertices in the first set.

The second line contains an integer n , representing the number of vertices in the second set.

The third line contains an integer e , representing the number of edges in the graph.

The next e lines each contain two integers u and v , representing an edge between vertex u in the first set and vertex v in the second set.

Output Format

The output prints the maximum matching pairs in increasing order of u , where each pair is printed in the format: (u, v)

If there are no matching pairs, print: "No matching found"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

3

5

1 1

2 2

4 3

3 3

4 1

Output: (1, 1)

(2, 2)

(3, 3)

Answer

```
// You are using GCC#include <stdio.h>
```

```
int m, n;
```

```
int adj[11][11];
```

```
int matchV[11];
```

```
int visited[11];
```

```
int dfs(int u) {
```

```
    for (int v = 1; v <= n; v++) {
```

```
        if (adj[u][v] && !visited[v]) {
```

```
            visited[v] = 1;
```

```
            if (matchV[v] == 0 || dfs(matchV[v])) {
```

```
                matchV[v] = u;
```

```
                return 1;
```

```
            }
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

```

int main() {
    int e;
    scanf("%d", &m);
    scanf("%d", &n);
    scanf("%d", &e);

    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++)
            adj[i][j] = 0;

    for (int i = 1; i <= n; i++)
        matchV[i] = 0;

    int u, v;

    for (int i = 0; i < e; i++) {
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
    }

    // maximum matching
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++)
            visited[j] = 0;

        if (dfs(i)) {
            // success
        }
    }

    int found = 0;

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (matchV[j] == i) {
                printf("(%d, %d)\n", i, j);
                found = 1;
            }
        }
    }
}

```

```
    if (!found) {  
        printf("No matching found");  
    }  
  
    return 0;  
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Alex needs to pair students from two different classes for a project, aiming to maximize the number of compatible pairs. Each student can be paired with only one student from the other class. The compatibility between students is represented as a binary adjacency matrix.

Alex wants to write a program to find the maximum number of compatible student pairs that can be formed.

Input Format

The first line contains an integer n , representing the number of students in each group.

The second line contains an integer m , representing the number of students in the second group.

The next n lines contain m space-separated integers each, representing the adjacency matrix of student compatibility.

Output Format

The single line containing the phrase "Maximum Bipartite Matching: x " where x is an integer representing the maximum number of compatible pairs that can be formed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

3

1 1 0

1 0 1

0 1 0

1 0 0

Output: Maximum Bipartite Matching: 3

Answer

// You are using GCC#include <stdio.h>

int n, m;

int adj[105][105];

int matchR[105];

int visited[105];

int dfs(int u) {

for (int v = 0; v < m; v++) {

if (adj[u][v] && !visited[v]) {

visited[v] = 1;

if (matchR[v] == -1 || dfs(matchR[v])) {

matchR[v] = u;

return 1;

}

}

}

return 0;

}

int main() {

scanf("%d", &n);

scanf("%d", &m);

// read adjacency matrix

for (int i = 0; i < n; i++)

for (int j = 0; j < m; j++)

scanf("%d", &adj[i][j]);

// initialize

for (int i = 0; i < m; i++)

matchR[i] = -1;

```

int result = 0;

// maximum matching
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++)
        visited[j] = 0;

    if (dfs(i))
        result++;
}

printf("Maximum Bipartite Matching: %d", result);

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement:

John is developing a recommendation system for a social networking platform. The system suggests potential friends to users based on their interests and mutual connections. To enhance the user experience, you want to ensure that the recommendation algorithm considers the removal of certain users from the network due to various reasons such as account deletion or user activity.

However, he still wants to maintain effective friend suggestions even after removing these users. John plan to integrate a functionality that calculates the maximum matching in the bipartite graph representing the user network after removing specific users and their connections.

Help him to achieve the task.

Input Format

Enter the number of vertices in partition

Enter the number of vertices in partition

Enter the adjacency matrix representation of the bipartite graph

Enter the vertex to remove

Output Format

Maximum matching after removing vertex

Sample Test Case

Input: 3

2

0 1

1 0

0 0

1

Output: Maximum matching after removing vertex 1:1

Answer

```
// You are using GCC#include <stdio.h>
```

```
int n, m;  
int adj[11][11];  
int matchR[11];  
int visited[11];  
int removeV;
```

```
int dfs(int u) {  
    if (u == removeV) return 0; // skip removed vertex
```

```
    for (int v = 0; v < m; v++) {  
        if (adj[u][v] && !visited[v]) {  
            visited[v] = 1;
```

```
            if (matchR[v] == -1 || dfs(matchR[v])) {  
                matchR[v] = u;  
                return 1;
```

```
            }  
        }  
    }  
    return 0;
```

```
}
```

```

int main() {
    scanf("%d", &n);
    scanf("%d", &m);

    // read matrix
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &adj[i][j]);

    scanf("%d", &removeV);

    // initialize
    for (int i = 0; i < m; i++)
        matchR[i] = -1;

    int result = 0;

    for (int i = 0; i < n; i++) {
        if (i == removeV) continue;

        for (int j = 0; j < m; j++)
            visited[j] = 0;

        if (dfs(i))
            result++;
    }

    printf("Maximum matching after removing vertex %d: %d", removeV, result);

    return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 11_COD

Attempt : 1

Total Mark : 30

Marks Obtained : 20

Section 1 : Coding

1. Problem Statement

Alice needs to assign works to employees in a company. Each employee can be assigned at most one work, and each work can be assigned to at most one employee. Her goal is to maximize the number of employees who are assigned works, ensuring that each work is assigned to exactly one employee and each employee is assigned at most one work.

Help Alice to complete the task.

Input Format

The first line consists of an integer N, representing the number of connections between employees and works.

The next N lines consists of two space-separated integers X and Y, representing a connection where X is the index of an employee (0-based indexing), and Y is

the index of a work (0-based indexing), indicating that employee X can perform work Y.

Output Format

The first line of the output displays "Maximum number of employees that could get works are: a", where a is an integer representing the maximum number of employees that could be assigned works according to the given conditions.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 9

0 0
0 1
0 2
1 0
2 1
3 4
4 3
4 4
5 5

Output: Maximum number of employees that could get works are: 6

Answer

```
// You a#include <stdio.h>
```

```
int adj[20][20]; // adjacency matrix
int match[20];   // work -> employee
int visited[20];
int maxEmp = 0, maxWork = 0;
```

```
int dfs(int emp) {
    for (int w = 0; w <= maxWork; w++) {
        if (adj[emp][w] && !visited[w]) {
            visited[w] = 1;

            if (match[w] == -1 || dfs(match[w])) {
                match[w] = emp;
                return 1;
            }
        }
    }
    return 0;
}
```

```

    }
    }
    }
    return 0;
}

```

```

int main() {
    int N;
    scanf("%d", &N);

    int x, y;

    // initialize adjacency
    for (int i = 0; i < 20; i++)
        for (int j = 0; j < 20; j++)
            adj[i][j] = 0;

    // read connections
    for (int i = 0; i < N; i++) {
        scanf("%d %d", &x, &y);
        adj[x][y] = 1;

        if (x > maxEmp) maxEmp = x;
        if (y > maxWork) maxWork = y;
    }
}

```

```

// initialize matching
for (int i = 0; i < 20; i++)
    match[i] = -1;

```

```

int result = 0;

```

```

for (int emp = 0; emp <= maxEmp; emp++) {
    for (int i = 0; i < 20; i++)
        visited[i] = 0;

    if (dfs(emp))
        result++;
}

```

```

printf("Maximum number of employees that could get works are: %d", result);

```

```
return 0;  
}re using GCC
```

Status : Wrong

Marks : 0/10

2. Problem Statement

Given an undirected bipartite graph with X vertices on the left side and Y vertices on the right side, the graph may contain multiple edges between the same pair of vertices.

Your task is to determine the minimum vertex cover for this graph.

A vertex cover of a graph is a set of vertices such that every edge in the graph is incident to at least one vertex in the set.

Input Format

The first line contains two space-separated integers X and Y , representing the number of vertices in the left set and right set, respectively.

The second line contains an integer e , representing the number of edges in the bipartite graph.

The next e lines each contain two space-separated integers u and v , representing an edge between: vertex u in the left set and vertex v in the right set

Output Format

The output prints the vertices in the minimum vertex cover, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4 4
4
0 1

0 2
1 3
2 3

Output: 0 3

Answer

```
#include <stdio.h>
```

```
int X, Y;  
int adj[100][100];  
int matchR[100];  
int visited[100];
```

```
// Maximum Matching
```

```
int bpm(int u) {  
    for (int v = 0; v < Y; v++) {  
        if (adj[u][v] && !visited[v]) {  
            visited[v] = 1;  
  
            if (matchR[v] == -1 || bpm(matchR[v])) {  
                matchR[v] = u;  
                return 1;  
            }  
        }  
    }  
    return 0;  
}
```

```
// DFS for alternating paths
```

```
void dfs(int u, int visL[], int visR[]) {  
    visL[u] = 1;  
  
    for (int v = 0; v < Y; v++) {  
        if (adj[u][v]) {  
            if (!visR[v]) {  
                visR[v] = 1;  
  
                if (matchR[v] != -1 && !visL[matchR[v]]) {  
                    dfs(matchR[v], visL, visR);  
                }  
            }  
        }  
    }  
}
```

```

    }
}

int main() {
    int e;
    scanf("%d %d", &X, &Y);
    scanf("%d", &e);

    // initialize
    for (int i = 0; i < X; i++)
        for (int j = 0; j < Y; j++)
            adj[i][j] = 0;

    for (int i = 0; i < Y; i++)
        matchR[i] = -1;

    int u, v;

    // input edges
    for (int i = 0; i < e; i++) {
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
    }

    // maximum matching
    for (int i = 0; i < X; i++) {
        for (int j = 0; j < Y; j++)
            visited[j] = 0;

        bpm(i);
    }

    // find matched left vertices
    int matchedL[100] = {0};
    for (int j = 0; j < Y; j++) {
        if (matchR[j] != -1) {
            matchedL[matchR[j]] = 1;
        }
    }

    int visL[100] = {0}, visR[100] = {0};

```

```

// DFS from unmatched left vertices
for (int i = 0; i < X; i++) {
    if (!matchedL[i]) {
        dfs(i, visL, visR);
    }
}

// print result
// Left NOT visited
for (int i = 0; i < X; i++) {
    if (!visL[i])
        printf("%d ", i);
}

// Right visited
for (int i = 0; i < Y; i++) {
    if (visR[i])
        printf("%d ", i);
}

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

You are given an undirected graph with 'num_vertices' vertices and 'num_edges' edges. Your task is to determine whether the given graph is bipartite or not. A graph is bipartite if it can be divided into two disjoint sets of vertices such that each edge connects a vertex from one set to a vertex from the other set.

Input Format

The first line contains two integers separated by a space: 'num_vertices' and 'num_edges'. These represent the number of vertices and edges in the graph, respectively.

The next 'num_edges' lines each contain two integers 'u' and 'v' separated by a space, representing an edge between vertex 'u' and vertex 'v'.

Output Format

The output displays a single line indicating whether the given graph is bipartite or not.

If the graph is bipartite, print "The graph is bipartite."

If the graph is not bipartite, print "The graph is not bipartite."

Sample Test Case

Input: 4 4

0 1

1 2

2 3

3 0

Output: The graph is bipartite.

Answer

```
#include <stdio.h>
```

```
int adj[100][100];
```

```
int color[100];
```

```
int n, e;
```

```
int isBipartite(int start) {
```

```
    int queue[100], front = 0, rear = 0;
```

```
    queue[rear++] = start;
```

```
    color[start] = 0;
```

```
    while (front < rear) {
```

```
        int u = queue[front++];
```

```
        for (int v = 0; v < n; v++) {
```

```
            if (adj[u][v]) {
```

```
                if (color[v] == -1) {
```

```
                    color[v] = 1 - color[u];
```

```
                    queue[rear++] = v;
```

```
                }
```

```
                else if (color[v] == color[u]) {
```

```
                    return 0;
```

```
                }
```

```

    }
    }
    }
    return 1;
}

int main() {
    scanf("%d %d", &n, &e);

    for (int i = 0; i < n; i++) {
        color[i] = -1;
        for (int j = 0; j < n; j++)
            adj[i][j] = 0;
    }

    int u, v;

    for (int i = 0; i < e; i++) {
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
        adj[v][u] = 1;
    }

    for (int i = 0; i < n; i++) {
        if (color[i] == -1) {
            if (!isBipartite(i)) {
                printf("The graph is not bipartite.");
                return 0;
            }
        }
    }

    printf("The graph is bipartite.");

    return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 11_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 15

Section 1 : Coding

1. _____ is a matching with the largest number of edges.

Answer

Maximum bipartite matching

Status : Correct

Marks : 1/1

2. How many colors are used in a bipartite graph?

Answer

2

Status : Correct

Marks : 1/1

3. What is the simplest method to prove that a graph is bipartite?

Answer

It does not have a cycle of an odd length

Status : Correct

Marks : 1/1

4. A matching that matches all the vertices of a graph is called?

Answer

Perfect matching

Status : Correct

Marks : 1/1

5. In a bipartite graph $G=(V,U,E)$, the matching of a free vertex in V to a free vertex in U called?

Answer

Augmenting

Status : Correct

Marks : 1/1

6. There are four students in a class namely A, B, C and D. A tells that a triangle is a bipartite graph. B tells pentagon is a bipartite graph. C tells square is a bipartite graph. D tells heptagon is a bipartite graph. Who among the following is correct?

Answer

C

Status : Correct

Marks : 1/1

7. Which of the following is not a property of the bipartite graph?

Answer

Asymmetric spectrum

Status : Correct

Marks : 1/1

8. The maximum number of edges in a bipartite graph with 14 vertices is _____.

Answer

49

Status : Correct

Marks : 1/1

9. What does the Halting Problem attempt to determine?

Answer

Whether a program will eventually stop running or continue forever

Status : Correct

Marks : 1/1

10. The Halting Problem was introduced by which computer scientist?

Answer

Alan Turing

Status : Correct

Marks : 1/1

11. Why is the Halting Problem considered unsolvable?

Answer

Because no algorithm can determine for every program whether it halts or runs forever

Status : Correct

Marks : 1/1

12. In computation theory, what does an undecidable problem mean?

Answer

A problem that cannot be solved by any algorithm for all inputs

Status : Correct

Marks : 1/1

13. In the context of the Halting Problem, what does the term halting refer to?

Answer

A program stopping its execution after completing its task

Status : Correct

Marks : 1/1

14. What is the main implication of the Halting Problem for computer science?

Answer

Some computational problems cannot be solved by any algorithm

Status : Correct

Marks : 1/1

15. The Halting Problem generally takes which of the following as input?

Answer

A program and its input data

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 9_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given an adjacency matrix $adj[][]$ of an undirected graph consisting of N vertices, find a Hamiltonian Path in the graph if one exists. If found, print the path as a sequence of vertex indices separated by $\rightarrow$. Otherwise, print No Hamiltonian Path found.

A Hamiltonian path visits each and every vertex of the graph exactly once. The adjacency matrix is symmetric with zeros on the diagonal (no self-loops).

Write a program using Backtracking to find the Hamiltonian Path.

Examples:

Input: $adj[][] = \{\{0, 1, 1, 1, 0\}, \{1, 0, 1, 0, 1\}, \{1, 1, 0, 1, 1\}, \{1, 0, 1, 0, 0\}\}$

Output: Yes

Explanation:

There exists a Hamiltonian Path for the given graph, as shown in the image below:

Input: $\text{adj}[][] = \{\{0, 1, 0, 0\}, \{1, 0, 1, 1\}, \{0, 1, 0, 0\}, \{0, 1, 0, 0\}\}$

Output: No

Input Format

The first line contains an integer N, representing the number of vertices in the graph.

The next N lines each contain N space-separated integers (0 or 1), representing the adjacency matrix of the graph.

Output Format

If a Hamiltonian path exists, print the vertices of the path separated by -> on a single line.

If no Hamiltonian path exists, print: No Hamiltonian Path found

Refer to the sample output for formatting specifications.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

0 1 1 1 0

1 0 1 0 1

1 1 0 1 1

1 0 1 0 0

0 1 1 0 0

Output: 0 -> 1 -> 4 -> 2 -> 3

Answer

```
#include <stdio.h>
```

```
int n;  
int graph[10][10];  
int path[10];  
int visited[10];  
  
int found = 0;  
  
int isSafe(int v, int pos) {  
    if (graph[path[pos - 1]][v] == 0)  
        return 0;  
  
    if (visited[v])  
        return 0;  
  
    return 1;  
}  
  
void hamiltonian(int pos) {  
    if (pos == n) {  
        found = 1;  
  
        for (int i = 0; i < n; i++) {  
            printf("%d", path[i]);  
            if (i != n - 1)  
                printf(" -> ");  
        }  
        return;  
    }  
  
    for (int v = 1; v < n; v++) {  
        if (isSafe(v, pos)) {  
            path[pos] = v;  
            visited[v] = 1;  
  
            hamiltonian(pos + 1);  
  
            if (found)  
                return;  
  
            visited[v] = 0;  
        }  
    }  
}
```

```

    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    path[0] = 0;
    visited[0] = 1;

    hamiltonian(1);

    if (!found) {
        printf("No Hamiltonian Path found");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

You are given an undirected graph with N vertices. Implement a function `longest_hamiltonian_cycle` to find the longest Hamiltonian cycle in the graph. A Hamiltonian cycle is a cycle that visits every vertex exactly once, except for the starting and ending vertices, which are the same.

Input Format

The first line of input consists of an integer N representing the number of vertices in the graph.

The next N lines each contain N space-separated integers, representing the adjacency matrix of the graph. The value at `graph[i][j]` is 1 if there is an edge

between vertex i and vertex j, and 0 otherwise.

Output Format

If a Hamiltonian cycle exists, print "Longest Hamiltonian Cycle:" followed by the vertices of the longest cycle found in order within square bracket separated by a comma.

If no Hamiltonian cycle exists, print "No Hamiltonian Cycle found".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 1 1 0

1 0 1 1

1 1 0 1

0 1 1 0

Output: Longest Hamiltonian Cycle: [0, 1, 3, 2]

Answer

```
#include <stdio.h>
```

```
int n;
```

```
int graph[10][10];
```

```
int path[10];
```

```
int visited[10];
```

```
int found = 0;
```

```
int isSafe(int v, int pos) {
```

```
    if (graph[path[pos - 1]][v] == 0)
```

```
        return 0;
```

```
    if (visited[v])
```

```
        return 0;
```

```
    return 1;
```

```
}
```

```
void solve(int pos) {
```

```

if (pos == n) {
    // check last to first connection
    if (graph[path[pos - 1]][path[0]] == 1) {
        found = 1;

        printf("Longest Hamiltonian Cycle: [");
        for (int i = 0; i < n; i++) {
            printf("%d", path[i]);
            if (i != n - 1)
                printf(", ");
        }
        printf("]");
    }
    return;
}

```

```

for (int v = 1; v < n; v++) {
    if (isSafe(v, pos)) {
        path[pos] = v;
        visited[v] = 1;

        solve(pos + 1);

        if (found)
            return;
    }
}

```

```

    visited[v] = 0;
}
}
}

```

```

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    path[0] = 0;
}

```

```
visited[0] = 1;
solve(1);

if (!found) {
    printf("No Hamiltonian Cycle found");
}

return 0;
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Nira is studying the N-Queens problem and needs to write a program to solve it. The program should find a valid arrangement of N queens on an N×N chessboard, ensuring that no two queens can attack each other.

Two queens attack each other if they are in the same row, the same column, or the same diagonal. Once a solution is found, display the board with queens placed correctly. A solution is guaranteed to exist for all valid values of N.

Write a program using Backtracking to solve the N-Queens problem.

Example

Input:

4 // Value of N

Output:

0 0 1 0

1 0 0 0

0 0 0 1

0 1 0 0

Explanation: The output represents a valid arrangement of queens on the 4x4 chessboard, where 1 indicates the position of a queen and 0 indicates an empty cell.

In the first row, a queen is placed in the third column. In the second row, a queen is placed in the first column. In the third row, a queen is placed in the fourth column. In the fourth row, a queen is placed in the second column. This arrangement of queens on the board ensures that no two queens threaten each other.

Input Format

The input consists of a single integer N, representing the size of the chessboard (N×N).

Output Format

Print the N×N board as N rows of N space-separated integers. Each integer is followed by a space (including the last integer on each row). Print 1 where a queen is placed and 0 for an empty cell. Print any one valid solution.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

Output: 0 0 1 0

1 0 0 0

0 0 0 1

0 1 0 0

Answer

```
#include <stdio.h>
```

```
int n;
```

```
int board[20][20];
```

```
int found = 0;
```

```
int isSafe(int row, int col) {
```

```
    for (int i = 0; i < row; i++) {
```

```
        if (board[i][col] == 1)
```

```

    return 0;
}

for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--) {
    if (board[i][j] == 1)
        return 0;
}

for (int i = row - 1, j = col + 1; i >= 0 && j < n; i--, j++) {
    if (board[i][j] == 1)
        return 0;
}

return 1;
}

```

```

void solve(int row) {
    if (row == n) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                printf("%d ", board[i][j]);
            }
            printf("\n");
        }
        found = 1;
        return;
    }
}

```

```

// ? CHANGE HERE
for (int col = n - 1; col >= 0; col--) {
    if (isSafe(row, col)) {
        board[row][col] = 1;

        solve(row + 1);

        if (found) return;

        board[row][col] = 0;
    }
}
}
}

```

```
int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            board[i][j] = 0;

    solve(0);

    return 0;
}
```

Status : Wrong

Marks : 0/10

4. Problem Statement

In chess, a knight moves in an "L" shape: two squares in one direction and one square perpendicular to it. From position (0, 0) on an infinite chessboard, a knight can move to one of the following eight positions in a single step:

(1, 2), (-1, 2), (1, -2), (-1, -2), (2, 1), (-2, 1), (2, -1), (-2, -1).

Given a target position (x, y) where both x and y are positive integers, find the minimum number of steps needed for the knight to travel from (0, 0) to (x, y) using BFS (Breadth-First Search).

Beginning from location (0,0), what is the minimum number of steps needed for a knight to get to some other arbitrary location (x,y)?

Input Format

The first line contains two space-separated integers x and y, representing the target position of the knight on the chessboard.

Output Format

Print a single integer representing the minimum number of steps needed for the knight to move from (0, 0) to (x, y).

Refer to the sample output for formatting specifications

Sample Test Case

Input: 1 2

Output: 1

Answer

```
// You are using GCC#include <stdio.h>
```

```
int dx[8] = {1, -1, 1, -1, 2, -2, 2, -2};  
int dy[8] = {2, 2, -2, -2, 1, 1, -1, -1};
```

```
int main() {  
    int x, y;  
    scanf("%d %d", &x, &y);
```

```
    int visited[20][20] = {0};
```

```
    // queue arrays  
    int qx[400], qy[400], dist[400];  
    int front = 0, rear = 0;
```

```
    // start from (0,0)  
    qx[rear] = 0;  
    qy[rear] = 0;  
    dist[rear] = 0;  
    rear++;
```

```
    visited[0][0] = 1;
```

```
    while (front < rear) {  
        int cx = qx[front];  
        int cy = qy[front];  
        int cd = dist[front];  
        front++;
```

```
        // reached target  
        if (cx == x && cy == y) {  
            printf("%d", cd);  
            return 0;
```

```

    }
    // explore 8 moves
    for (int i = 0; i < 8; i++) {
        int nx = cx + dx[i];
        int ny = cy + dy[i];

        // limit search space (safe bound)
        if (nx >= -10 && ny >= -10 && nx <= 10 && ny <= 10) {
            if (!visited[nx + 10][ny + 10]) {
                visited[nx + 10][ny + 10] = 1;

                qx[rear] = nx;
                qy[rear] = ny;
                dist[rear] = cd + 1;
                rear++;
            }
        }
    }
}

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

A customer is shopping in a supermarket and has selected N items, each with a fixed price.

The customer has a discount coupon that can be applied only if the total price of some selected items is exactly equal to a target amount S .

Write a program using Backtracking to find all distinct subsets of item prices whose sum is exactly equal to S .

Each item price can be used only once. If the input contains duplicate prices, the output must not contain repeated subsets.

Formula: A subset is valid if: $p_1 + p_2 + \dots + p_k = S$

Print each valid subset with its elements in ascending order. Print subsets in the order they are discovered by the backtracking algorithm (which processes the sorted array from left to right).

Input Format

The first line contains an integer N representing the number of items.

The second line contains N space-separated integers representing item prices. The prices may include duplicate values.

The third line contains an integer S representing the target bill amount.

Output Format

Print each distinct valid subset on a separate line. Within each subset, print elements in ascending order, each followed by a space (including the last element).

Print subsets in the order they are found during backtracking on the sorted input.

If no valid subset exists, print: No subset found

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
5 5 10 15 20
25
Output: 5 5 15
5 20
10 15

Answer

```
// You are using GCC#include <stdio.h>
```

```
int n, S;  
int arr[20], subset[20];  
int found = 0;
```

```

// sort array
void sort() {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (arr[i] > arr[j]) {
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
}

```

```

void backtrack(int start, int sum, int k) {
    if (sum == S) {
        for (int i = 0; i < k; i++) {
            printf("%d ", subset[i]);
        }
        printf("\n");
        found = 1;
        return;
    }
}

```

```

for (int i = start; i < n; i++) {

```

```

    // skip duplicates
    if (i > start && arr[i] == arr[i - 1])
        continue;

```

```

    if (sum + arr[i] > S)
        break;

```

```

    subset[k] = arr[i];

```

```

    backtrack(i + 1, sum + arr[i], k + 1);
}

```

```

int main() {
    scanf("%d", &n);
    for (int i = 0; i < n; i++)

```

```
scanf("%d", &arr[i]);  
scanf("%d", &S);  
sort();  
backtrack(0, 0, 0);  
  
if (!found) {  
    printf("No subset found");  
}  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 9_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 41

Section 1 : Coding

1. Problem Statement

Diya is given a weighted, undirected graph with N vertices. She wants to find the minimum weight Hamiltonian cycle in the graph, if one exists. A Hamiltonian cycle is a cycle that visits every vertex exactly once, except for the starting and ending vertices, which are the same. The weight of a Hamiltonian cycle is the sum of the weights of all edges in the cycle:

$$W = w(v_1, v_2) + w(v_2, v_3) + \dots + w(v_n, v_1)$$

Write a program using Backtracking to find the minimum weight Hamiltonian cycle.

Example: For a graph with 4 vertices and the adjacency matrix:

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

The algorithm explores all Hamiltonian cycles starting from vertex 0.

Among all valid cycles, 0 1 3 2 0 has the minimum weight: $10 + 25 + 30 + 15 = 80$.

Input Format

The first line contains an integer N representing the number of vertices in the graph.

The next N lines each contain N space-separated integers representing the adjacency matrix of the weighted graph. The value at $\text{graph}[i][j]$ is the weight of the edge between vertices i and j. A value of 0 means no edge exists. The matrix is symmetric ($\text{graph}[i][j] == \text{graph}[j][i]$) and the diagonal is always 0 ($\text{graph}[i][i] == 0$).

Output Format

If a minimum weight Hamiltonian cycle exists, print Minimum Weight Hamiltonian Cycle: on the first line, then print the vertices of the cycle in square brackets separated by a comma and a space, including the starting vertex at both the beginning and the end of the cycle. On the next line, print Total Weight: followed by a space and the integer total weight.

If no Hamiltonian cycle exists, print No Hamiltonian Cycle found with minimum weight.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output: Minimum Weight Hamiltonian Cycle:

[0, 1, 3, 2, 0]

Total Weight: 80

Answer

```
#include <stdio.h>
```

```
#define MAX 10
```

```
#define INF 99999
```

```
int n;
```

```
int graph[MAX][MAX];
```

```
int visited[MAX];
```

```
int path[MAX], bestPath[MAX];
```

```
int minCost = INF;
```

```
void tsp(int pos, int count, int cost) {
```

```
    if (count == n && graph[pos][0] != 0) {  
        int totalCost = cost + graph[pos][0];
```

```
        if (totalCost < minCost) {  
            minCost = totalCost;  
            for (int i = 0; i < n; i++)  
                bestPath[i] = path[i];
```

```
    }  
    return;
```

```
}  
for (int i = 0; i < n; i++) {  
    if (!visited[i] && graph[pos][i] != 0) {  
        visited[i] = 1;  
        path[count] = i;
```

```
        tsp(i, count + 1, cost + graph[pos][i]);
```

```
        visited[i] = 0; // backtrack
```

```
    }  
}
```

```
int main() {
```

```
    scanf("%d", &n);
```

```

for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        scanf("%d", &graph[i][j]);

for (int i = 0; i < n; i++)
    visited[i] = 0;

visited[0] = 1;
path[0] = 0;

tsp(0, 1, 0);

if (minCost == INF) {
    printf("No Hamiltonian Cycle found with minimum weight.");
} else {
    printf("Minimum Weight Hamiltonian Cycle:\n");
    printf("[");

    for (int i = 0; i < n; i++) {
        printf("%d", bestPath[i]);
        printf(", ");
    }

    printf("0]\n");
    printf("Total Weight: %d", minCost);
}

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Alice is tasked with determining whether a given graph contains a Hamiltonian Cycle and, if it does, listing all such cycles. A Hamiltonian Cycle in a graph is a cycle that visits each vertex exactly once and returns to the starting vertex.

Write a program using Backtracking that takes a graph represented by an adjacency matrix and finds all Hamiltonian Cycles starting from the first vertex (vertex 0), if any exist.

Input Format

The first line contains an integer N representing the number of vertices in the graph.

The next N lines each contain a single string – the name of each vertex (no surrounding spaces).

The following N lines each contain N space-separated integers (0 or 1) representing the adjacency matrix. A value of 1 at position $[i][j]$ means an edge exists between vertex i and vertex j; 0 means no edge.

Output Format

For each Hamiltonian Cycle found, print a line in the format: `['v0', 'v1', ..., 'vn', 'v0']` where vertex names are enclosed in single quotes, separated by a comma and a space, with the starting vertex repeated at the end. Print cycles in the order they are discovered by the backtracking algorithm.

If no Hamiltonian Cycle exists, print: No Hamiltonian Cycle

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 8

a
b
c
d
e
f
g
h


```
0 1 0 0 0 0 0 1
1 0 1 0 0 0 0 0
0 1 0 1 0 0 0 1
0 0 1 0 1 0 1 0
0 0 0 1 0 1 0 0
0 0 0 0 1 0 1 0
0 0 0 1 0 1 0 1
1 0 1 0 0 0 1 0
```

Output: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'a']
['a', 'h', 'g', 'f', 'e', 'd', 'c', 'b', 'a']

Answer

```
// You are using GCC#include <stdio.h>
#include <string.h>
```

```
#define MAX 10
```

```
int n;
int graph[MAX][MAX];
int visited[MAX];
int path[MAX];
char names[MAX][20];
int found = 0;
```

```
void printCycle() {
    printf("[");
    for (int i = 0; i < n; i++) {
        printf("%s", names[path[i]]);
    }
    printf("%s]\n", names[path[0]]);
}
```

```
void hamiltonian(int pos, int count) {
    // If all vertices visited
    if (count == n) {
        // check edge back to start
        if (graph[pos][0] == 1) {
            printCycle();
            found = 1;
        }
        return;
    }
}
```

```

    for (int i = 1; i < n; i++) { // start from 1 (0 already used)
        if (!visited[i] && graph[pos][i] == 1) {
            visited[i] = 1;
            path[count] = i;

            hamiltonian(i, count + 1);

            visited[i] = 0; // backtrack
        }
    }
}

```

```

int main() {
    scanf("%d", &n);

    // Read vertex names (IMPORTANT: preserve spaces)
    for (int i = 0; i < n; i++) {
        scanf(" %[^\\n]", names[i]);
    }

    // Read adjacency matrix
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);

    // Initialize
    for (int i = 0; i < n; i++)
        visited[i] = 0;

    visited[0] = 1;
    path[0] = 0;

    hamiltonian(0, 1);

    if (!found) {
        printf("No Hamiltonian Cycle");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

A company has N employees, and each employee can contribute a fixed number of training hours.

The manager wants to form groups of employees such that the total training hours of a selected group is exactly equal to the required training goal S.

Write a program using Backtracking to find all distinct subsets of employee training hours whose sum is exactly equal to S.

Each employee's training hours can be used only once.

If the input contains duplicate hour values, the output must not contain repeated subsets.

Condition

A subset is valid if:

$$h_1 + h_2 + \dots + h_k = S$$

Input Format

The first line contains an integer N representing the number of employees.

The second line contains N space-separated integers representing the training hours of each employee. The input may contain duplicate values.

The third line contains an integer S representing the required training goal.

Output Format

Print each distinct valid subset on a separate line. Within each subset, print elements in ascending order, each followed by a space (including the last element). Print subsets in the order they are found during backtracking on the sorted input.

If no valid subset exists, print: No subset found

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
2 2 3 5 6 8
10

Output: 2 2 6
2 3 5
2 8

Answer

```
// You are using GCC#include <stdio.h>
```

```
int n, S;  
int arr[20], subset[20];  
int found = 0;
```

```
// Simple sort
```

```
void sort() {  
    for (int i = 0; i < n - 1; i++) {  
        for (int j = i + 1; j < n; j++) {  
            if (arr[i] > arr[j]) {  
                int temp = arr[i];  
                arr[i] = arr[j];  
                arr[j] = temp;  
            }  
        }  
    }  
}
```

```
void backtrack(int start, int sum, int k) {  
    if (sum == S) {  
        for (int i = 0; i < k; i++) {  
            printf("%d ", subset[i]);  
        }  
        printf("\n");  
        found = 1;  
        return;  
    }  
}
```

```
for (int i = start; i < n; i++) {
```

```

// skip duplicates
if (i > start && arr[i] == arr[i - 1])
    continue;

if (sum + arr[i] > S)
    break;

subset[k] = arr[i];
backtrack(i + 1, sum + arr[i], k + 1);
}
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    scanf("%d", &S);

    sort();

    backtrack(0, 0, 0);

    if (!found) {
        printf("No subset found");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Nikila is exploring the N Queens Problem. This classic problem involves placing N queens on an $N \times N$ chessboard such that no two queens threaten each other. In chess, queens can move horizontally, vertically, and diagonally, so the challenge is to arrange the queens to ensure they do not

share the same row, column, or diagonal.

Write a program using Backtracking to find and print all valid solutions to the N Queens Problem.

Input Format

The input consists of a single integer N representing both the size of the chessboard (N×N) and the number of queens to be placed.

Output Format

Print all valid solutions in the order they are found by the backtracking algorithm. Each solution is represented as an N×N matrix where 1 indicates the position of a queen and 0 represents an empty cell. Each value is followed by a space. Each row is printed on a new line. Print a blank line after each solution including the last one. If no valid solution exists, print nothing.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

Output: 0 0 1 0

1 0 0 0

0 0 0 1

0 1 0 0

0 1 0 0

0 0 0 1

1 0 0 0

0 0 1 0

Answer

```
// You are using GCC#include <stdio.h>
```

```
int n;  
int board[10][10];
```

```
// Check if safe
```

```

int isSafe(int row, int col) {
    // Check column
    for (int i = 0; i < row; i++) {
        if (board[i][col] == 1)
            return 0;
    }

    // Check left diagonal
    for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--) {
        if (board[i][j] == 1)
            return 0;
    }

    // Check right diagonal
    for (int i = row - 1, j = col + 1; i >= 0 && j < n; i--, j++) {
        if (board[i][j] == 1)
            return 0;
    }

    return 1;
}

```

```

// Backtracking
void solve(int row) {
    if (row == n) {
        // Print solution
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                printf("%d ", board[i][j]);
            }
            printf("\n");
        }
        printf("\n"); // blank line after each solution
        return;
    }
}

```

```

for (int col = 0; col < n; col++) {
    if (isSafe(row, col)) {
        board[row][col] = 1;

        solve(row + 1);
    }
}

```

```

        board[row][col] = 0; // backtrack
    }
}
}

```

```

int main() {
    scanf("%d", &n);

    // initialize board
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            board[i][j] = 0;

    solve(0);

    return 0;
}

```

Status : Partially correct

Marks : 1/10

5. Problem Statement

Sarah wants to verify whether a given arrangement of queens on a 4×4 chessboard constitutes a valid solution to the 4 Queens Problem. The objective is to position four queens on the board in a way that prevents any queen from threatening another, considering horizontal, vertical, and diagonal movements.

To determine the validity of the solution, Sarah needs to ensure that there are exactly four queens on the board, each row and column contains only one queen, and no two queens share the same diagonal.

Your task is to assist Sarah by implementing a program that checks these criteria for the given chessboard arrangement.

Input Format

The input consists of 4 lines, each containing 4 space-separated integers representing one row of the chessboard. Each integer is either 1 (queen present) or 0 (empty cell).

Output Format

Print a single line indicating whether the placement is valid, checking conditions in the following priority order:

1. If the total number of queens is not exactly 4: False - Invalid number of queens: X where X is the count of queens on the board.
2. If a row does not contain exactly one queen: False - Row Y has Z queens where Y is the 0-based row index and Z is the number of queens in that row.
3. If a column does not contain exactly one queen: False - Column Y has Z queens where Y is the 0-based column index and Z is the number of queens in that column.
4. If a main diagonal (top-left to bottom-right) contains more than one queen: False - Main diagonal (d=X) has Y queens where X is the diagonal identifier computed as column - row (range -3 to 3) and Y is the number of queens on that diagonal.
5. If an anti-diagonal (top-right to bottom-left) contains more than one queen: False - Anti-diagonal (d=X) has Y queens where X is the diagonal identifier computed as column + row (range 0 to 6) and Y is the number of queens on that diagonal.
6. If all conditions are satisfied: True - Valid 4 Queens solution: All conditions satisfied

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0

Output: True - Valid 4 Queens solution: All conditions satisfied

Answer

```
// You are using GCC#include <stdio.h>
```

```
int main() {  
    int a[4][4];  
    int total = 0;  
  
    // input  
    for (int i = 0; i < 4; i++) {
```

```
for (int j = 0; j < 4; j++) {  
    scanf("%d", &a[i][j]);  
    if (a[i][j] == 1)  
        total++;  
}  
}
```

```
// 1. total queens check  
if (total != 4) {  
    printf("False - Invalid number of queens: %d", total);  
    return 0;  
}
```

```
// 2. row check  
for (int i = 0; i < 4; i++) {  
    int count = 0;  
    for (int j = 0; j < 4; j++) {  
        if (a[i][j] == 1)  
            count++;  
    }  
    if (count != 1) {  
        printf("False - Row %d has %d queens", i, count);  
        return 0;  
    }  
}
```

```
// 3. column check  
for (int j = 0; j < 4; j++) {  
    int count = 0;  
    for (int i = 0; i < 4; i++) {  
        if (a[i][j] == 1)  
            count++;  
    }  
    if (count != 1) {  
        printf("False - Column %d has %d queens", j, count);  
        return 0;  
    }  
}
```

```
// 4. main diagonal check (col - row)  
for (int d = -3; d <= 3; d++) {  
    int count = 0;
```

```

    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            if (j - i == d && a[i][j] == 1)
                count++;
        }
    }
    if (count > 1) {
        printf("False - Main diagonal (d=%d) has %d queens", d, count);
        return 0;
    }
}

// 5. anti-diagonal check (col + row)
for (int d = 0; d <= 6; d++) {
    int count = 0;
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            if (i + j == d && a[i][j] == 1)
                count++;
        }
    }
    if (count > 1) {
        printf("False - Anti-diagonal (d=%d) has %d queens", d, count);
        return 0;
    }
}

// valid case
printf("True - Valid 4 Queens solution: All conditions satisfied");

return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 9_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Bob has a task to find a Hamiltonian cycle in a given undirected simple graph (no self-loops) represented as an adjacency matrix, where a value of 1 indicates an edge exists between two vertices and a value of 0 indicates no edge. A Hamiltonian cycle is a cycle that visits every vertex exactly once and returns to the starting vertex. Bob needs to write a program that will determine if such a cycle exists and, if so, print the cycle starting and ending at vertex 0. If no cycle is found, the program should output "No solution".

Your task is to help Bob implement this.

Input Format

The first line contains an integer V, representing the number of vertices.

The following V lines each contain V space-separated integers (0 or 1), representing the adjacency matrix. The matrix is symmetric (undirected graph) and has zeros on the diagonal (no self-loops)

Output Format

If a Hamiltonian cycle is found, the first line prints "Solution found" and the second line prints the cycle as a series of space-separated integers, representing the vertices starting and ending at vertex 0.

If no Hamiltonian cycle is found, print "No solution".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 8

0 1 0 1 1 0 0 0

1 0 1 0 0 1 0 0

0 1 0 1 0 0 1 0

1 0 1 0 0 0 0 1

1 0 0 0 0 1 0 1

0 1 0 0 1 0 1 0

0 0 1 0 0 1 0 1

0 0 0 1 1 0 1 0

Output: Solution found

0 1 2 3 7 6 5 4 0

Answer

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int graph[MAX][MAX];
```

```
int path[MAX];
```

```
int V;
```

```
int isSafe(int v, int pos) {
```

```
    if(graph[path[pos - 1]][v] == 0)
```

```
        return 0;
```

```

    for(int i = 0; i < pos; i++) {
        if(path[i] == v)
            return 0;
    }

    return 1;
}

int solve(int pos) {
    if(pos == V) {
        return graph[path[pos - 1]][path[0]] == 1;
    }

    for(int v = 1; v < V; v++) {
        if(isSafe(v, pos)) {
            path[pos] = v;

            if(solve(pos + 1))
                return 1;

            path[pos] = -1;
        }
    }

    return 0;
}

int main() {
    scanf("%d", &V);

    for(int i = 0; i < V; i++) {
        for(int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    for(int i = 0; i < V; i++) {
        path[i] = -1;
    }

    path[0] = 0;

```

```

if(!solve(1)) {
    printf("No solution");
    return 0;
}

printf("Solution found\n");

for(int i = 0; i < V; i++) {
    printf("%d ", path[i]);
}
printf("0");

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Given an undirected complete graph with N vertices ($N \geq 3$), find the number of distinct Hamiltonian cycles in the graph using backtracking. A Hamiltonian cycle is a cycle that visits each vertex exactly once and returns to the starting vertex.

Two Hamiltonian cycles are considered identical if one can be obtained from the other by rotation (changing the starting vertex) or reversal (traversing in the opposite direction).

Example:

For $N=4$, the vertices are 0, 1, 2, 3. The 3 distinct Hamiltonian cycles are:

0 1 2 3 00 1 3 2 00 2 1 3 0

Output: 3

Input Format

The input consists of a single integer N , representing the number of vertices in the complete graph.

Output Format

Print a single integer on its own line representing the number of distinct Hamiltonian cycles in the complete graph with N vertices.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

Output: 60

Answer

```
// You are using GCC#include <stdio.h>
```

```
int main() {  
    int N;  
    scanf("%d", &N);  
  
    long long fact = 1;  
  
    // Calculate (N-1)!  
    for(int i = 2; i <= N - 1; i++) {  
        fact *= i;  
    }  
  
    // Divide by 2  
    printf("%lld", fact / 2);  
  
    return 0;  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

A college is organizing an event and has collected donations from N sponsors. Each sponsor contributes a fixed amount. The event coordinator wants to know all possible groups of sponsors whose donation sum

matches exactly the required event budget S.

Write a program using Backtracking to find all subsets of sponsor contributions whose sum is exactly equal to S.

A subset may contain any number of sponsors, but each sponsor amount can be used only once.

Formula:

A subset is valid if: $a_1 + a_2 + \dots + a_k = S$

Input Format

The first line contains an integer N, representing the number of sponsors.

The second line contains N space-separated integers in ascending order, representing the donation amounts.

The third line contains an integer S, representing the required budget.

Output Format

Print all subsets whose sum is exactly S, one subset per line. Within each subset, print the elements in the order they appear in the input, each followed by a space. Subsets are printed in the order they are discovered during backtracking (subsets starting with smaller elements appear first).

If no subset exists, print: No subset found

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
10 20 30 40
50

Output: 10 40
20 30

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int arr[25];
int subset[25];
int N, S;
int found = 0;
```

```
// Backtracking function
void solve(int index, int count, int sum) {
```

```
    // If sum matches
    if(sum == S) {
        found = 1;
        for(int i = 0; i < count; i++) {
            printf("%d ", subset[i]);
        }
        printf("\n");
        return;
    }
```

```
    // If out of bounds or sum exceeded
    if(index >= N || sum > S)
        return;
```

```
    // Include current element
    subset[count] = arr[index];
    solve(index + 1, count + 1, sum + arr[index]);
```

```
    // Exclude current element
    solve(index + 1, count, sum);
}
```

```
int main() {
    scanf("%d", &N);
```

```
    for(int i = 0; i < N; i++) {
        scanf("%d", &arr[i]);
    }
```

```
    scanf("%d", &S);
```

```
solve(0, 0, 0);  
  
if(!found) {  
    printf("No subset found");  
}  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Emily is working on arranging queens on a chessboard where one queen must already be fixed at a given position. She wants to find all possible configurations of placing N queens such that no two queens attack each other, with the condition that the queen at the specified position cannot be moved.

A queen can attack any cell in its row, column, or diagonal. Row and column indices are 0-based.

Write a program to ensure that the chessboard solutions are displayed correctly, or state "No solution" if none exist.

Input Format

The first line contains an integer N representing the size of the chessboard.

The second line contains an integer i, representing the row index of the fixed queen.

The third line contains an integer j, representing the column index of the fixed queen.

Output Format

The first line of output consists of a single floating-point number representing the maximum total value, formatted to two decimal places.

The next n lines each consist of a single floating-point number representing the

fraction x_i of each item selected, formatted to two decimal places. x_i represents the fraction of the i -th item taken (1.00 means the item is fully taken, a value between 0.00 and 1.00 means a fraction is taken). The fractions should be printed in the same order as the input items.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0

1

Output: . Q . .

. . . Q

Q . . .

. . Q .

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int N;
```

```
int board[MAX][MAX];
```

```
int fixedRow, fixedCol;
```

```
int found = 0;
```

```
// Check if safe
```

```
int isSafe(int row, int col) {
```

```
    // Check column
```

```
    for(int i = 0; i < row; i++)
```

```
        if(board[i][col])
```

```
            return 0;
```

```
    // Check left diagonal
```

```
    for(int i=row, j=col; i>=0 && j>=0; i--, j--)
```

```
        if(board[i][j])
```

```
            return 0;
```

```

// Check right diagonal
for(int i=row, j=col; i>=0 && j<N; i--, j++)
    if(board[i][j])
        return 0;

return 1;
}

// Backtracking
void solve(int row) {
    if(row == N) {
        found = 1;

        // Print board
        for(int i = 0; i < N; i++) {
            for(int j = 0; j < N; j++) {
                if(board[i][j])
                    printf("Q ");
                else
                    printf(". ");
            }
            printf("\n");
        }
        printf("\n");
        return;
    }

    // If fixed row, only place at fixed column
    if(row == fixedRow) {
        if(isSafe(row, fixedCol)) {
            board[row][fixedCol] = 1;
            solve(row + 1);
            board[row][fixedCol] = 0;
        }
        return;
    }

    // Try all columns
    for(int col = 0; col < N; col++) {
        if(isSafe(row, col)) {
            board[row][col] = 1;

```

```
        solve(row + 1);  
        board[row][col] = 0;  
    }  
}  
}
```

```
int main() {  
    scanf("%d", &N);  
    scanf("%d", &fixedRow);  
    scanf("%d", &fixedCol);  
  
    // Initialize board  
    for(int i = 0; i < N; i++)  
        for(int j = 0; j < N; j++)  
            board[i][j] = 0;  
  
    solve(0);  
  
    if(!found) {  
        printf("No solution");  
    }  
  
    return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 9_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 14

Section 1 : MCQ

1. The problem of finding a path in a graph that visits every vertex exactly once is called?

Answer

Hamiltonian path problem

Status : Correct

Marks : 1/1

2. In the Hamiltonian Cycle problem, if a graph contains a Hamiltonian cycle, it is called a

Answer

Hamiltonian graph

Status : Correct

Marks : 1/1

3. What is the primary difficulty in finding a Hamiltonian Path in a general graph?

Answer

It is an NP-complete problem.

Status : Correct

Marks : 1/1

4. Which of the following statements is true regarding the backtracking algorithm for the Hamiltonian Cycle problem?

Answer

It explores different paths in the graph to find a Hamiltonian cycle.

Status : Correct

Marks : 1/1

5. Which algorithmic paradigm is commonly used to solve the Hamiltonian Cycle problem?

Answer

Backtracking

Status : Correct

Marks : 1/1

6. The following paradigm can be used to find the solution to the problem in minimum time: Given a set of non-negative integers, and a value K, determine if there is a subset of the given set with a sum equal to K:

Answer

Dynamic Programming

Status : Correct

Marks : 1/1

7. What is a subset sum problem?

Answer

Checking for the presence of a subset that has the sum of elements equal to a given number and printing true or false based on the result

Status : Correct

Marks : 1/1

8. Which of the following is not true about the subset sum problem?

Answer

the dynamic programming solution has a time complexity of $O(n \log n)$

Status : Correct

Marks : 1/1

9. What is the worst case time complexity of dynamic programming solution of the subset sum problem(sum=given subset sum)?

Answer

$O(\text{sum} * n)$

Status : Correct

Marks : 1/1

10. Placing N-queens so that no two queens attack each other is called _____.

Answer

N-queen's problem

Status : Correct

Marks : 1/1

11. Where is the n-queens problem implemented?

Answer

Chess

Status : Correct

Marks : 1/1

12. Which of the following methods can be used to solve N-Queen's problem?

Answer

Backtracking

Status : Correct

Marks : 1/1

13. Which one of the following is an application of the backtracking algorithm?

Answer

Crossword

Status : Correct

Marks : 1/1

14. For N Queens problem, the time complexity is

Answer

$O(N!)$

Status : Correct

Marks : 1/1

15. How many unique solutions are there to the 4 Queens problem, where solutions that are reflections or rotations of each other are considered the same?

Answer

1

Status : Wrong

Marks : 0/1

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 7_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Mary is planning to build a railway network to connect cities efficiently. She needs a program that, given the number of cities and the segments between them, determines the minimum cost to connect all cities using an undirected weighted graph.

Implement a solution using Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of the city network. If it is not possible to connect all cities, output an appropriate message

Example

Input

3

3

0 1 4

1 2 5

0 2 3

Output

7

Explanation

The input specifies 3 cities and 3 track segments with their corresponding weights.

The track segments are:

Track Segment 1: Connects city 0 and city 1, with a weight of 4.

Track Segment 2: Connects city 1 and city 2 with a weight of 5.

Track Segment 3: Connects city 0 and city 2 with a weight of 3.

The algorithm finds the minimum cost to build railway tracks connecting all cities. In this case, the minimum spanning tree includes track segment 3 (0-2 with weight 3) and track segment 1 (0-1 with weight 4). The total cost is $3 + 4 = 7$.

Input Format

The first line of input contains an integer `num_cities`, representing the number of cities (numbered 0 to `num_cities - 1`).

The second line contains an integer `num_segments`, representing the number of undirected segments (edges) between cities.

The next `num_segments` lines each contain three space-separated integers: source, destination, and weight, representing an undirected segment between city source and city destination with the given cost.

Output Format

The first line of output prints an integer representing the minimum cost to connect all cities.

If it is not possible to connect all the cities, the output prints 'It is not possible to connect all cities' (without a period).

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

3

0 1 4

1 2 5

0 2 3

Output: 7

Answer

```
// You are using GCC#include <stdio.h>
```

```
#define MAX 100
```

```
struct Edge {  
    int u, v, w;  
};
```

```
struct Edge edges[MAX];  
int parent[MAX];
```

```
// Find (with path compression)
```

```
int find(int x) {  
    if(parent[x] != x)  
        parent[x] = find(parent[x]);  
    return parent[x];  
}
```

```
// Union
```

```
void unionSet(int a, int b) {  
    int pa = find(a);  
    int pb = find(b);  
    parent[pa] = pb;  
}
```

```
// Sort edges by weight (bubble sort)
```

```
void sortEdges(int m) {  
    for(int i = 0; i < m - 1; i++) {  
        for(int j = 0; j < m - i - 1; j++) {  
            if(edges[j].w > edges[j + 1].w) {  
                struct Edge temp = edges[j];  
                edges[j] = edges[j + 1];  
                edges[j + 1] = temp;  
            }  
        }  
    }  
}
```

```
int main() {  
    int n, m;  
    scanf("%d", &n);  
    scanf("%d", &m);
```

```
    // Read edges  
    for(int i = 0; i < m; i++) {  
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);  
    }
```

```
    // Initialize parent  
    for(int i = 0; i < n; i++) {  
        parent[i] = i;  
    }
```

```
    // Sort edges  
    sortEdges(m);
```

```
    int totalCost = 0;  
    int count = 0;
```

```
    // Kruskal  
    for(int i = 0; i < m; i++) {  
        int u = edges[i].u;  
        int v = edges[i].v;
```

```
        if(find(u) != find(v)) {  
            unionSet(u, v);  
            totalCost += edges[i].w;
```

```

        count++;
    }
}

// Check if MST possible
if(count == n - 1) {
    printf("%d", totalCost);
} else {
    printf("It is not possible to connect all cities");
}

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Maya, a project planner, must connect multiple office branches using network cables while keeping the total cost within a given budget. Each possible connection between two branches has an associated cost.

Write a program to find the Minimum Spanning Tree (MST) of the branch network using Kruskal's Algorithm on an undirected weighted graph. If the total cost of the MST is within the given budget, display the cost. Otherwise, indicate that it is not possible.

It is guaranteed that the graph is connected in all test cases

Input Format

The first line contains three space-separated integers V, E, and budget, representing the number of branches (vertices, numbered 1 to V), the number of possible connections (edges), and the total budget respectively.

Each of the next E lines contains three space-separated integers src, dest, and weight, representing an undirected connection between branch src and branch dest with cost weight.

Output Format

If the MST cost is within the budget, the output prints:

'Minimum Cost Spanning Tree within the budget: X'

where X is an integer representing the total minimum cost.

Otherwise, the output prints:

'A valid MST within the budget is not possible.'

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3 3 5

1 2 2

2 3 3

3 1 4

Output: Minimum Cost Spanning Tree within the budget: 5

Answer

```
#include <stdio.h>
```

```
#define MAX 1000
```

```
struct Edge {  
    int u, v, w;  
};
```

```
struct Edge edges[MAX];  
int parent[105];
```

```
// Find  
int find(int x) {  
    if(parent[x] != x)  
        parent[x] = find(parent[x]);  
    return parent[x];  
}
```

```
// Union  
void unionSet(int a, int b) {
```



```
int pa = find(a);
int pb = find(b);
parent[pa] = pb;
}
```

```
// Sort edges
void sortEdges(int m) {
    for(int i = 0; i < m - 1; i++) {
        for(int j = 0; j < m - i - 1; j++) {
            if(edges[j].w > edges[j + 1].w) {
                struct Edge temp = edges[j];
                edges[j] = edges[j + 1];
                edges[j + 1] = temp;
            }
        }
    }
}
```

```
int main() {
    int V, E;
    long long budget;

    scanf("%d %d %lld", &V, &E, &budget);

    // Read edges
    for(int i = 0; i < E; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }

    // Initialize parent (1-based)
    for(int i = 1; i <= V; i++) {
        parent[i] = i;
    }

    sortEdges(E);

    long long totalCost = 0;
    int count = 0;

    // Kruskal
    for(int i = 0; i < E; i++) {
        int u = edges[i].u;
```

```

int v = edges[i].v;

if(find(u) != find(v)) {
    unionSet(u, v);
    totalCost += edges[i].w;
    count++;
}
}

if(totalCost <= budget) {
    printf("Minimum Cost Spanning Tree within the budget: %lld", totalCost);
} else {
    printf("A valid MST within the budget is not possible.");
}

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Sita is planning to connect multiple locations in her city with roads. Each possible road between two locations has a construction distance (in kilometres). The city is represented as an undirected weighted graph using an adjacency matrix, where a value of 0 indicates no direct road between two locations (locations are numbered 0 to numLocations - 1).

To minimise the overall construction distance, she decides to use Prim's Algorithm to construct the Minimum Spanning Tree (MST). Help her find the MST edges and the total minimum distance.

Input Format

The first line of input contains an integer numLocations, representing the number of locations (numbered 0 to numLocations - 1).

The next numLocations lines each contain numLocations space-separated integers representing the adjacency matrix, where graph[i][j] is the distance between location i and location j. A value of 0 indicates no direct road between those two locations.

Output Format

The first line of output prints: 'Location\tDistance'

Each of the next numLocations - 1 lines prints the MST edge in the format:

'parent -> child \t\t\t dist km'

where parent is the parent location number, child is the child location number, and dist is the edge weight (integer) in kilometres.

The last line of output prints: 'Minimum total distance: totalCost km'

where totalCost is an integer.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2

0 5

5 0

Output: Location Distance

0 -> 1 5 km

Minimum total distance: 5 km

Answer

```
#include <stdio.h>
```

```
#define MAX 10
```

```
#define INF 99999
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int graph[MAX][MAX];
```

```

// Read adjacency matrix
for(int i = 0; i < n; i++) {
    for(int j = 0; j < n; j++) {
        scanf("%d", &graph[i][j]);
        if(graph[i][j] == 0 && i != j)
            graph[i][j] = INF; // no edge
    }
}

```

```

int parent[MAX];
int key[MAX];
int visited[MAX] = {0};

```

```

// Initialize
for(int i = 0; i < n; i++) {
    key[i] = INF;
}

```

```

key[0] = 0;
parent[0] = -1;

```

```

// Prim's Algorithm
for(int count = 0; count < n - 1; count++) {
    int min = INF, u = -1;

```

```

    for(int i = 0; i < n; i++) {
        if(!visited[i] && key[i] < min) {
            min = key[i];
            u = i;
        }
    }
}

```

```

visited[u] = 1;

```

```

for(int v = 0; v < n; v++) {
    if(!visited[v] && graph[u][v] < key[v]) {
        key[v] = graph[u][v];
        parent[v] = u;
    }
}
}

```

```

int totalCost = 0;

// Output
printf("Location\tDistance\n");

for(int i = 1; i < n; i++) {
    printf("%d -> %d\t\t\t%d km\n", parent[i], i, graph[parent[i]][i]);
    totalCost += graph[parent[i]][i];
}

printf("Minimum total distance: %d km", totalCost);

return 0;
}

```

Status : Wrong

Marks : 0/10

4. Problem Statement

Riya works at a data tools company that needs a compact representation of frequently used characters for fast transmission. She decides to use Huffman Coding, a lossless compression technique that assigns shorter bit-strings to more frequent symbols and longer bit-strings to less frequent ones.

The Huffman codes should be printed in the order that leaf nodes are encountered during a left-to-right traversal of the constructed Huffman tree.

Write a program that builds a Huffman tree from a list of symbols with their frequencies and prints the Huffman codes for each symbol.

Input Format

The first line contains an integer n , representing the number of distinct symbols.

Each of the next n lines contains a single printable character (the symbol) followed by a space and a positive integer (the frequency), representing the symbol and its occurrence count.

Output Format

The output prints n lines, one per symbol, in the order that leaf nodes are encountered during a left-to-right traversal of the Huffman tree (left child before right child at every internal node).

Each line prints the symbol followed by a colon, a space, and its Huffman code (a binary string of 0s and 1s):

'<symbol>: <binary_code>'

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2

a 1

b 2

Output: a: 0

b: 1

Answer

```
#include <iostream>
#include <cstring>
using namespace std;
```

```
#define MAX 300
```

```
struct Node {
    char ch;
    int freq;
    Node *left, *right;
};
```

```
Node* createNode(char ch, int freq) {
    Node* node = new Node();
    node->ch = ch;
```

```

    node->freq = freq;
    node->left = node->right = NULL;
    return node;
}

```

```

void findMin(Node* arr[], int n, int *min1, int *min2) {
    *min1 = -1;
    *min2 = -1;

    for(int i = 0; i < n; i++) {
        if(arr[i] == NULL) continue;

        if(*min1 == -1 || arr[i]->freq < arr[*min1]->freq) {
            *min2 = *min1;
            *min1 = i;
        } else if(*min2 == -1 || arr[i]->freq < arr[*min2]->freq) {
            *min2 = i;
        }
    }
}

```

```

void generateCodes(Node* root, char* code, int depth) {
    if(root == NULL) return;

    if(root->left == NULL && root->right == NULL) {
        code[depth] = '\0';
        cout << root->ch << ": " << code << "\n";
        return;
    }

    code[depth] = '0';
    generateCodes(root->left, code, depth + 1);

    code[depth] = '1';
    generateCodes(root->right, code, depth + 1);
}

```

```

int main() {
    int n;
    cin >> n;

    Node* arr[MAX];
}

```

```

char ch;
int freq;

for(int i = 0; i < n; i++) {
    cin >> ch >> freq;
    arr[i] = createNode(ch, freq);
}

int size = n;

while(size > 1) {
    int min1, min2;
    findMin(arr, size, &min1, &min2);

    Node* left = arr[min1];
    Node* right = arr[min2];

    Node* newNode = createNode('$', left->freq + right->freq);
    newNode->left = left;
    newNode->right = right;

    arr[min1] = newNode;
    arr[min2] = arr[size - 1];
    size--;
}

Node* root = arr[0];

char code[100];
generateCodes(root, code, 0);

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

In a data science class, Professor Huffman demonstrates how encoded messages can be decoded using a pre-built Huffman tree.

Given a Huffman code table (mapping each character to its binary code) and an encoded binary message, decode the message back to its original text by traversing the Huffman tree.

The code table is guaranteed to be a valid prefix-free Huffman code.

The encoded message consists only of '0' and '1' characters, and every bit maps to exactly one symbol in the Huffman tree

The tree is defined as follows:

Internal nodes are represented by "*". The left edge (0) leads to the symbol "A". The right edge (1) leads to the symbol "B".

This means:

"0" A "1" B

Your task is to decode a given encoded message (a string of 0s and 1s) back into its original form consisting of "A" and "B".

Input Format

The first line contains an integer n , representing the number of symbols in the code table.

Each of the next n lines contains a character and its Huffman code (binary string), separated by a space.

The last line contains the encoded binary message (a string of 0s and 1s) to be decoded

Output Format

The output prints the decoded message as a sequence of characters ('A' or 'B'), followed by a newline.

Refer to the sample output for formatting specifications

Sample Test Case

Input: 11100011100

Output: BBBAABBBA

Answer

```
#include <iostream>
using namespace std;

int main() {
    string encoded;
    cin >> encoded;

    for(int i = 0; i < encoded.length(); i++) {
        if(encoded[i] == '0')
            cout << "A";
        else
            cout << "B";
    }

    return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 7_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 42.5

Section 1 : Coding

1. Problem Statement

Bob is working on a project involving network optimization. He needs a program to find the Minimum Spanning Tree (MST) of a given undirected weighted graph with 5 nodes (numbered 0 to 4).

The graph is represented as a 5×5 symmetric adjacency matrix, where a value of 0 indicates no direct connection between two nodes. Implement Prim's Algorithm to find the MST and print each edge with its weight.

Example

Input

0 3 0 5 0

4 0 2 3 5

0 3 0 8 7

3 8 0 0 9

0 5 7 2 0

Output

Edge Weight

0 - 1 4

1 - 2 3

1 - 3 8

1 - 4 5

Explanation

In the given 5x5 matrix representing an undirected graph, Bob has specified the weights of edges. The Prim's algorithm is applied to find the Minimum Spanning Tree (MST). The output displays the edges along with their corresponding weights, forming the optimal tree connecting all vertices. In this case, the MST includes edges (0-1), (1-2), (1-3), and (1-4) with weights 4, 3, 8, and 5 respectively, ensuring minimal total weight for network optimization.

Input Format

The input consists of 5 lines, each containing 5 space-separated integers, representing the adjacency matrix of the graph. A value of 0 indicates no direct connection between two nodes. The matrix is symmetric ($\text{graph}[i][j] = \text{graph}[j][i]$) since the graph is undirected.

Output Format

The first line of output prints the header: 'Edge \tWeight'

Each of the next 4 lines prints one MST edge in the format:

'u - v \tw'

where u is the parent node, v is the child node, and w is the integer edge weight, printed in ascending order of child node index (node-index order).

Refer to the sample output for formatting specifications.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 0 2 0 6 0

2 0 3 8 5

0 3 0 8 7

6 8 0 0 9

0 5 7 9 0

Output: Edge Weight

0 - 1 2

1 - 2 3

0 - 3 6

1 - 4 5

Answer

```
#include <stdio.h>
```

```
#define V 5
```

```
#define INF 99999
```

```
int main() {
```

```
    int graph[V][V];
```

```
    // Input
```

```
    for(int i = 0; i < V; i++) {
```

```
        for(int j = 0; j < V; j++) {
```

```
            scanf("%d", &graph[i][j]);
```

```
        }
```

```
    }
```

```
    int parent[V];
```

```
    int key[V];
```

```
    int visited[V] = {0};
```

```

// Initialize
for(int i = 0; i < V; i++) {
    key[i] = INF;
}

key[0] = 0;
parent[0] = -1;

// Prim's Algorithm
for(int count = 0; count < V - 1; count++) {
    int min = INF, u = -1;

    for(int i = 0; i < V; i++) {
        if(!visited[i] && key[i] < min) {
            min = key[i];
            u = i;
        }
    }

    visited[u] = 1;

    for(int v = 0; v < V; v++) {
        if(graph[u][v] != 0 && !visited[v] && graph[u][v] < key[v]) {
            parent[v] = u;
            key[v] = graph[u][v];
        }
    }
}

// Output
printf("Edge \tWeight\n");
for(int i = 1; i < V; i++) {
    printf("%d - %d \t%d\n", parent[i], i, graph[parent[i]][i]);
}

return 0;
}

```

Status : Partially correct

Marks : 7.5/10

2. Problem Statement

You are given a set of cities in a country and the cost of building railway track segments to connect them. Your task is to find the minimum cost required to build the railway tracks and connect all the cities. The cost of each track segment is given, along with the source city and destination city to which it connects.

Write a program that takes the number of cities, the number of track segments, and the details of each track segment as input. Implement Kruskal's Algorithm to calculate the minimum cost of building the railway tracks.

Cities are numbered 0 to N-1.

Example

Input:

3

3

0 1 4

1 2 5

0 2 3

Output:

7

Explanation:

The input specifies 3 cities and 3 track segments with their corresponding weights.

The track segments are:

Track Segment 1: Connects city 0 and city 1, with a weight of 4.

Track Segment 2: Connects city 1 and city 2 with a weight of 5.

Track Segment 3: Connects city 0 and city 2 with a weight of 3. The algorithm finds the

minimum cost to build railway tracks connecting all cities. In this case, the minimum spanning tree includes track segment 3 (0-2 with weight 3) and track segment 1 (0-1 with weight 4). The total cost is $3 + 4 = 7$.

Input Format

The first line of input contains an integer N, representing the number of cities (numbered 0 to N-1).

The second line contains an integer M, representing the number of track segments.

Each of the next M lines contains three space-separated integers: the source city, the destination city, and the weight (cost) of the track segment

Output Format

The first line of input contains an integer N, representing the number of cities (numbered 0 to N-1).

The second line contains an integer M, representing the number of track segments.

Each of the next M lines contains three space-separated integers: the source city, the destination city, and the weight (cost) of the track segment

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

5

0 1 1

0 2 4

1 2 2

1 3 5

2 3 3

Output: 6

Answer

```
#include <stdio.h>
```



```
#define MAX 100
```

```
struct Edge {  
    int u, v, w;  
};
```

```
struct Edge edges[MAX];  
int parent[MAX];
```

```
// Find function
```

```
int find(int x) {  
    if(parent[x] != x)  
        parent[x] = find(parent[x]);  
    return parent[x];  
}
```

```
// Union function
```

```
void unionSet(int a, int b) {  
    int pa = find(a);  
    int pb = find(b);  
    parent[pa] = pb;  
}
```

```
// Sort edges
```

```
void sortEdges(int m) {  
    for(int i = 0; i < m - 1; i++) {  
        for(int j = 0; j < m - i - 1; j++) {  
            if(edges[j].w > edges[j + 1].w) {  
                struct Edge temp = edges[j];  
                edges[j] = edges[j + 1];  
                edges[j + 1] = temp;  
            }  
        }  
    }  
}
```

```
int main() {  
    int N, M;  
    scanf("%d", &N);  
    scanf("%d", &M);
```

```

// Read edges
for(int i = 0; i < M; i++) {
    scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
}

// Initialize parent
for(int i = 0; i < N; i++) {
    parent[i] = i;
}

// Sort edges
sortEdges(M);

int totalCost = 0;
int edgeCount = 0;

// Kruskal
for(int i = 0; i < M; i++) {
    int u = edges[i].u;
    int v = edges[i].v;

    if(find(u) != find(v)) {
        unionSet(u, v);
        totalCost += edges[i].w;
        edgeCount++;
    }
}

// Output
if(edgeCount == N - 1) {
    printf("%d", totalCost);
} else {
    printf("It is not possible to connect all cities.");
}

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Raja is given a map of cities connected by bidirectional roads, where each road has an associated construction cost (weight). His task is to find the Minimum Spanning Tree (MST) of this network using Prim's Algorithm, starting from city 0, so that all cities are connected with the minimum total construction cost.

Prim's Algorithm works as follows:

Start from the source city (city 0). Mark it as included in the MST. At each step, select the edge with the minimum weight that connects a city already in the MST to a city not yet in the MST. Repeat until all cities are included in the MST.

The program should display each MST edge (the city included and the cost to connect it) and the total MST cost.

Note: The given graph is a connected, undirected, weighted graph. Self-loops and parallel edges are not present.

Input Format

The first line of input consists of two space-separated integers, n and m , where n represents the number of cities (vertices) and m represents the number of roads (edges).

The next m lines each consist of three space-separated integers u , v , and w , where u and v are the two cities connected by the road (0-indexed) and w is the construction cost (weight) of the road.

Output Format

The output consists of n lines.

The first $n-1$ lines each print the MST edge added, in the format:

"Edge: $u - v$, Cost: w ", where u is the city already in the MST, v is the newly added city, and w is the integer edge weight. Edges are printed in the order they are

added by Prim's algorithm starting from city 0.

The last line prints "Total MST Cost: t", where t is the total integer cost of the minimum spanning tree.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4 5

0 1 10

0 2 6

0 3 5

1 3 15

2 3 4

Output: Edge: 0 - 1, Cost: 10

Edge: 3 - 2, Cost: 4

Edge: 0 - 3, Cost: 5

Total MST Cost: 19

Answer

```
#include <stdio.h>
```

```
#define MAX 100
```

```
#define INF 999999
```

```
int main() {
```

```
    int n, m;
```

```
    scanf("%d %d", &n, &m);
```

```
    int graph[MAX][MAX];
```

```
    // Initialize
```

```
    for(int i = 0; i < n; i++) {
```

```
        for(int j = 0; j < n; j++) {
```

```
            graph[i][j] = INF;
```

```
        }
```

```
    }
```

```
    int u, v, w;
```

```

// Read edges
for(int i = 0; i < m; i++) {
    scanf("%d %d %d", &u, &v, &w);
    graph[u][v] = w;
    graph[v][u] = w;
}

int key[MAX], parent[MAX], visited[MAX] = {0};

for(int i = 0; i < n; i++) {
    key[i] = INF;
    parent[i] = -1;
}

key[0] = 0;

// Prim's Algorithm (standard)
for(int count = 0; count < n; count++) {
    int min = INF, u = -1;

    for(int i = 0; i < n; i++) {
        if(!visited[i] && key[i] < min) {
            min = key[i];
            u = i;
        }
    }

    visited[u] = 1;

    for(int v = 0; v < n; v++) {
        if(graph[u][v] != INF && !visited[v] && graph[u][v] < key[v]) {
            key[v] = graph[u][v];
            parent[v] = u;
        }
    }
}

int totalCost = 0;

// PRINT IN ORDER OF NODE INDEX (1 n-1)
for(int i = 1; i < n; i++) {

```

```

    printf("Edge: %d - %d, Cost: %d\n", parent[i], i, graph[parent[i]][i]);
    totalCost += graph[parent[i]][i];
}

printf("Total MST Cost: %d", totalCost);

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Riya is working as a data scientist in a company that deals with large text data. Due to storage and bandwidth limitations, she needs a way to compress text while maintaining its readability.

She decides to use Huffman Encoding, a well-known algorithm that assigns shorter binary codes to frequently occurring characters and longer binary codes to less common characters, thus reducing the overall message size.

Given a set of characters with their respective frequencies, help Riya generate Huffman Codes for each character. Then, using these codes, encode a given message.

Assign '0' to left branches and '1' to right branches, merge the two nodes with the lowest frequency at each step. Clarify that the message will only contain characters from the given input set. Specify that Huffman codes are printed in the order they appear during in-order/pre-order traversal of the Huffman tree (matching sample output)

The message will only contain characters from the provided character set.

Input Format

The first line of input consists of an integer n , representing the number of unique characters.

The next n lines each consist of a character (an uppercase or lowercase English

letter) and a positive integer representing its frequency, separated by a space.

The last line of input consists of a string representing the message to be encoded. The message will only contain characters from the provided character set.

Output Format

The first n lines of output print the Huffman code for each character in the format:

"character: code"

where the characters are printed in the order they appear as leaf nodes during the Huffman tree traversal (left child before right child, assigning '0' to left and '1' to right).

All n input characters and their codes are printed, regardless of whether they appear in the message.

The last line of output prints the encoded message as a sequence of 0s and 1s obtained by replacing each character in the message with its corresponding Huffman code.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

A 1

B 2

C 3

ABCCBAABBC

Output: C: 0

A: 10

B: 11

10110011101011110

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#define MAX 100
```

```
struct Node {  
    char ch;  
    int freq;  
    struct Node *left, *right;  
};
```

```
struct Node* createNode(char ch, int freq) {  
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));  
    node->ch = ch;  
    node->freq = freq;  
    node->left = node->right = NULL;  
    return node;  
}
```

```
void findMin(struct Node* arr[], int n, int *min1, int *min2) {  
    *min1 = -1;  
    *min2 = -1;
```

```
    for(int i = 0; i < n; i++) {  
        if(arr[i] == NULL) continue;
```

```
        if(*min1 == -1 || arr[i]->freq < arr[*min1]->freq) {  
            *min2 = *min1;  
            *min1 = i;  
        } else if(*min2 == -1 || arr[i]->freq < arr[*min2]->freq) {  
            *min2 = i;  
        }  
    }
```

```
}
```

```
// IMPORTANT: full declaration (your error was here)  
char codes[256][100];
```

```
void generateCodes(struct Node* root, char* code, int depth) {  
    if(root == NULL) return;
```

```
    if(root->left == NULL && root->right == NULL) {  
        code[depth] = '\0';
```



```

        strcpy(codes[root->ch], code);
        printf("%c: %s\n", root->ch, code);
        return;
    }

    code[depth] = '0';
    generateCodes(root->left, code, depth + 1);

    code[depth] = '1';
    generateCodes(root->right, code, depth + 1);
}

int main() {
    int n;
    scanf("%d", &n);

    struct Node* arr[MAX];

    char ch;
    int freq;

    for(int i = 0; i < n; i++) {
        scanf(" %c %d", &ch, &freq);
        arr[i] = createNode(ch, freq);
    }

    int size = n;

    while(size > 1) {
        int min1, min2;
        findMin(arr, size, &min1, &min2);

        struct Node* left = arr[min1];
        struct Node* right = arr[min2];

        struct Node* newNode = createNode('$', left->freq + right->freq);
        newNode->left = left;
        newNode->right = right;

        arr[min1] = newNode;
        arr[min2] = arr[size - 1];
        size--;
    }
}

```

```

    }

    struct Node* root = arr[0];

    char code[100];
    generateCodes(root, code, 0);

    char message[100];
    scanf("%s", message);

    for(int i = 0; message[i]; i++) {
        printf("%s", codes[(int)message[i]]);
    }

    return 0;
}

```

Status : Partially correct

Marks : 7.5/10

5. Problem Statement

In a clandestine world of spies and covert operations, a group of operatives uses Huffman coding to encode and decode their classified messages securely.

Huffman coding assigns shorter binary codes to more frequently occurring characters and longer codes to less frequent ones. The algorithm works as follows:

Each character and its frequency is inserted into a min-heap (priority queue). At each step, extract the two nodes with the lowest frequencies, merge them into a new internal node whose frequency is the sum of the two, and reinsert it into the heap. Repeat until only one node (the root) remains — this is the Huffman tree. Assign '0' to every left branch and '1' to every right branch. The binary string from root to each leaf node is that character's Huffman code.

Given a set of characters with their frequencies, build the Huffman tree, print the codes for all characters, encode the given message using these codes, and decode the encoded message back to the original.

Note: The message will only contain characters from the provided input set. All character frequencies are unique.

Input Format

The first line of input consists of a positive integer N, representing the number of unique characters.

The next N lines each consist of a character (an uppercase or lowercase English letter) and a positive integer representing its frequency, separated by a space.

The last line of input consists of a string representing the secret message to be encoded. The message contains only characters from the provided character set

Output Format

The first N lines of output each print the Huffman code for a character in the format:

"character: code"

where characters are printed in the order they appear as leaf nodes during Huffman tree traversal (left child before right child, with '0' assigned to left and '1' to right).

The next line prints the encoded message — a sequence of 0s and 1s obtained by replacing each character in the message with its Huffman code.

The last line prints the decoded message obtained by traversing the Huffman tree using the encoded bits, which must match the original input message.

Refer to the sample outputs for better understanding and formatting.

Sample Test Case

Input: 5

A 5

B 9

C 12
D 13
E 16
ABCDE
Output: C: 00
D: 01
A: 100
B: 101
E: 11
100101000111
ABCDE

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 100

struct Node {
    char ch;
    int freq;
    struct Node *left, *right;
};

struct Node* createNode(char ch, int freq) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->ch = ch;
    node->freq = freq;
    node->left = node->right = NULL;
    return node;
}

void findMin(struct Node* arr[], int n, int *min1, int *min2) {
    *min1 = -1;
    *min2 = -1;

    for(int i = 0; i < n; i++) {
        if(arr[i] == NULL) continue;
        if(*min1 == -1 || arr[i]->freq < arr[*min1]->freq) {
            *min2 = *min1;
```

```

        *min1 = i;
    } else if(*min2 == -1 || arr[i]->freq < arr[*min2]->freq) {
        *min2 = i;
    }
}
}
}

```

```

char codes[256][100];

```

```

void generateCodes(struct Node* root, char* code, int depth) {
    if(root == NULL) return;

```

```

    if(root->left == NULL && root->right == NULL) {
        code[depth] = '\0';
        strcpy(codes[(int)root->ch], code);
        printf("%c: %s\n", root->ch, code);
        return;
    }

```

```

    code[depth] = '0';
    generateCodes(root->left, code, depth + 1);

```

```

    code[depth] = '1';
    generateCodes(root->right, code, depth + 1);
}

```

```

int main() {
    int n;
    scanf("%d", &n);

```

```

    struct Node* arr[MAX];

```

```

    char ch;
    int freq;

```

```

    for(int i = 0; i < n; i++) {
        scanf(" %c %d", &ch, &freq);
        arr[i] = createNode(ch, freq);
    }

```

```

    int size = n;

```

```

while(size > 1) {
    int min1, min2;
    findMin(arr, size, &min1, &min2);

    struct Node* left = arr[min1];
    struct Node* right = arr[min2];

    struct Node* newNode = createNode('$', left->freq + right->freq);
    newNode->left = left;
    newNode->right = right;

    arr[min1] = newNode;
    arr[min2] = arr[size - 1]; // FIXED LINE
    size--;
}

struct Node* root = arr[0];

char code[100];
generateCodes(root, code, 0);

char message[100];
scanf("%s", message);

char encoded[1000] = "";
for(int i = 0; message[i]; i++) {
    strcat(encoded, codes[(int)message[i]]);
}

printf("%s\n", encoded);

struct Node* curr = root;
for(int i = 0; encoded[i]; i++) {
    if(encoded[i] == '0')
        curr = curr->left;
    else
        curr = curr->right;

    if(curr->left == NULL && curr->right == NULL) {
        printf("%c", curr->ch);
        curr = root;
    }
}

```

```
}  
return 0;  
}
```

Status : Partially correct

Marks : 7.5/10

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 7_COD

Attempt : 1

Total Mark : 30

Marks Obtained : 27.5

Section 1 : Coding

1. Problem Statement

Prim's Algorithm for Minimum Spanning Tree (MST):

Sita is planning to lay roads to connect 5 locations in her city. The distances between locations are given as an adjacency matrix, where a value of 0 indicates no direct road between two locations (locations are numbered 0 to 4).

Help her find the Minimum Spanning Tree (MST) using Prim's Algorithm so that all locations are connected with the minimum total road length. Print each MST edge and its weight in the order they are added to the MST.

Input Format

The input consists of 5 lines, each containing 5 space-separated integers,

representing the adjacency matrix of the graph. A value of 0 indicates no direct connection between two locations.

Output Format

The first line of output prints the header: 'Edge \tWeight'

Each of the next V-1 lines prints one MST edge in the format:

'u - v\tw'

where u is the parent location, v is the child location, and w is the integer edge weight, printed in the order edges are added to the MST (MST discovery order).

Refer to the sample output for formatting specifications.

Refer to the sample output for format specifications.

Sample Test Case

Input: 0 2 0 6 0

2 0 3 8 5

0 3 0 8 7

6 8 0 0 9

0 5 7 9 0

Output: Edge Weight

0 - 1 2

1 - 2 3

1 - 4 5

0 - 3 6

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define V 5
```

```
int minKey(int key[], int mstSet[]) {
```

```

int min = INT_MAX, min_index;

for (int v = 0; v < V; v++) {
    if (mstSet[v] == 0 && key[v] < min) {
        min = key[v];
        min_index = v;
    }
}
return min_index;
}

int main() {
    int graph[V][V];

    // Input adjacency matrix
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    int parent[V];    // To store MST
    int key[V];       // Minimum weights
    int mstSet[V];    // Visited array

    // Initialize
    for (int i = 0; i < V; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }

    key[0] = 0;    // Start from node 0
    parent[0] = -1;

    // Build MST
    for (int count = 0; count < V - 1; count++) {
        int u = minKey(key, mstSet);
        mstSet[u] = 1;

        for (int v = 0; v < V; v++) {
            if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
                parent[v] = u;
            }
        }
    }
}

```

```

        key[v] = graph[u][v];
    }
}

// Output
printf("Edge \tWeight\n");
for (int i = 1; i < V; i++) {
    printf("%d - %d\t%d\n", parent[i], i, graph[i][parent[i]]);
}

return 0;
}

```

Status : Partially correct

Marks : 7.5/10

2. Problem statement

Alex is a network engineer tasked with laying cables to connect multiple offices in the city. Each possible cable connection between two offices has an associated cost. To minimize the total cabling cost while ensuring all offices are connected, Alex decides to use Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of the office network.

Given a connected, undirected, weighted graph with V vertices (numbered 0 to $V-1$) and E edges, find the MST and print each edge included along with the total minimum cost.

Input Format

The first line contains an integer V , representing the number of vertices (numbered 0 to $V-1$).

The second line contains an integer E , representing the number of edges.

Each of the next E lines contains three space-separated integers u , v , and w , representing an undirected edge between vertex u and vertex v with weight w .

Output Format

The first line of output prints: 'Edges in the constructed MST:'

Each of the next V-1 lines prints one MST edge in the format:

'u -- v == weight'

where u and v are vertex numbers (integers) and weight is the edge cost (integer), printed in the order edges are added to the MST (ascending weight order).

The last line prints: 'Minimum Spanning Tree: totalCost'

where totalCost is an integer representing the total MST weight.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

3

0 1 2

1 2 2

0 2 2

Output: Edges in the constructed MST:

0 -- 1 == 2

1 -- 2 == 2

Minimum Spanning Tree: 4

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Edge {  
    int u, v, w;  
};
```

```
int parent[100];
```

```
// Find with path compression
int find(int i) {
    if (parent[i] != i)
        parent[i] = find(parent[i]);
    return parent[i];
}
```

```
// Union
void unionSet(int x, int y) {
    int rootX = find(x);
    int rootY = find(y);
    parent[rootX] = rootY;
}
```

```
// Comparator for sorting edges
int compare(const void* a, const void* b) {
    return ((struct Edge*)a)->w - ((struct Edge*)b)->w;
}
```

```
int main() {
    int V, E;
    scanf("%d", &V);
    scanf("%d", &E);

    struct Edge edges[E];

    for (int i = 0; i < E; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }
```

```
// Sort edges by weight
qsort(edges, E, sizeof(struct Edge), compare);
```

```
// Initialize disjoint set
for (int i = 0; i < V; i++) {
    parent[i] = i;
}
```

```
int count = 0;
int totalCost = 0;

printf("Edges in the constructed MST:\n");
```

```

for (int i = 0; i < E && count < V - 1; i++) {
    int u = edges[i].u;
    int v = edges[i].v;
    int w = edges[i].w;

    if (find(u) != find(v)) {
        printf("%d -- %d == %d\n", u, v, w);
        totalCost += w;
        unionSet(u, v);
        count++;
    }
}

printf("Minimum Spanning Tree: %d", totalCost);

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

You are designing a data compression tool that uses Huffman Coding to encode data efficiently.

Write a program to read a set of characters along with their corresponding Huffman variable-length binary codes, and use them to generate the encoded binary string for a given input message.

It is guaranteed that every character in the input message has a corresponding Huffman code in the provided code table.

Input Format

The first line contains an integer N (integer), representing the number of characters in the code table.

Each of the next N lines contains a character C and its corresponding Huffman code (a binary string of 0s and 1s), separated by a space.

The last line contains the input message (a string of characters) that needs to be encoded. Every character in the message is guaranteed to have a corresponding entry in the code table.

Output Format

The output prints 'Coded Data: ' followed by a binary string (a sequence of 0s and 1s) representing the encoded message, obtained by concatenating the Huffman code of each character in the input message in order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 10

D 0100

U 0000

F 0101

M 0001

A 0010

N 0011

C 1100

O 1101

E 1110

H 1111

HUFF

Output: Coded Data: 1111000001010101

Answer

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    int N;
```

```
    scanf("%d", &N);
```

```
    char ch;
```

```
    char code[26][25];
```

```
    // Initialize all codes
```

```
    for (int i = 0; i < 26; i++) {
```

```
code[i][0] = '\0';  
}  
  
// Read character-code pairs  
for (int i = 0; i < N; i++) {  
    scanf(" %c", &ch);    // read character (skip whitespace)  
    scanf("%s", code[ch - 'A']); // read its code  
}  
  
char message[105];  
scanf("%s", message);  
  
printf("Coded Data: ");  
  
// Encode message  
for (int i = 0; i < strlen(message); i++) {  
    printf("%s", code[message[i] - 'A']);  
}  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 7_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 14

Section 1 : MCQ

1. The frequencies of letters in a file are:

$s = 17, k = 24, y = 37$

A Huffman tree is constructed, and we assign 0 to the left branch and 1 to the right branch at each node.

Which of the following is the Huffman code for the letter 's'?

Answer

10

Status : Correct

Marks : 1/1

2. In Huffman coding, does data in a tree always occur?

Answer

Leaves

Status : Correct

Marks : 1/1

3. Which of the following algorithms is the best approach for solving Huffman codes?

Answer

Greedy algorithm

Status : Correct

Marks : 1/1

4. From the following given tree, what is the computed codeword for 'c'?

Answer

110

Status : Correct

Marks : 1/1

5. From the following given tree, what is the code word for the character 'a'?

Answer

011

Status : Correct

Marks : 1/1

6. Kruskal's algorithm is used to _____.

Answer

Find minimum spanning tree

Status : Correct

Marks : 1/1

7. Consider the following graph:

Which edges would be included in the minimum spanning tree using Prim's algorithm starting from vertex A?

Answer

AB, BD, DE, EF, FC

Status : Correct

Marks : 1/1

8. Which of the following is true?

Answer

Prim's algorithm initializes with a vertex

Status : Correct

Marks : 1/1

9. Prim's algorithm can be efficiently implemented using _____ for graphs with greater density.

Answer

fibonacci heap

Status : Wrong

Marks : 0/1

10. Which of the following is false about Prim's algorithm?

Answer

It constructs MST by selecting edges in increasing order of their weights

Status : Correct

Marks : 1/1

11. What is the worst-case time complexity of Prim's algorithm if the adjacency matrix is used?

Answer

$O(V^2)$

Status : Correct

Marks : 1/1

12. Consider a graph $G = (V, E)$, where $V = \{v_1, v_2, \dots, v_{100}\}$, $E = \{(v_i, v_j) \mid 1 \leq i < j \leq 100\}$ and the weight of the edge (v_i, v_j) is $i - j$. The weight of the minimum spanning tree of G is _____.

Answer

99

Status : Correct

Marks : 1/1

13. An undirected graph $G(V, E)$ contains n ($n > 2$) nodes named $v_1, v_2, \dots, v_n$. Two nodes v_i, v_j are connected if and only if $0 < |i - j| \leq 2$. Each edge (v_i, v_j) is assigned a weight $i + j$. A sample graph with $n = 4$ is shown below. What will be the cost of the minimum spanning tree (MST) of such a graph with n nodes?

Answer

$n^2 - n + 1$

Status : Correct

Marks : 1/1

14. An undirected graph G has n nodes. Its adjacency matrix is given by an $n \times n$ square matrix whose (i) diagonal elements are 0's and (ii) non-diagonal elements are 1's. which one of the following is TRUE?

Answer

Graph G has multiple distinct MSTs, each of cost $n-1$

Status : Correct

Marks : 1/1

15. Consider a weighted complete graph G on the vertex set $\{v_1, v_2, \dots, v_n\}$ such that the weight of the edge (v_i, v_j) is $2|i-j|$. The weight of a minimum spanning tree of G is:

Answer

$2n-2$

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 5_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Suppose you are working on a project that involves sorting a large dataset of student records based on a single criterion, such as their CGPA, age, or major. You have decided to use a modified quicksort algorithm to efficiently sort the dataset while ensuring that the sorting is stable, i.e., that records with equal values maintain their original relative order.

To achieve this, you will implement a modified version of the quicksort algorithm that maintains stability during the sorting process. Specifically, you will use a three-way partitioning scheme that sorts the records into three groups: those with values less than the pivot, those with values equal to the pivot, and those with values greater than the pivot. When the algorithm processes subarrays with equal pivot values, it uses insertion sort to maintain stability.

Write a function that implements this modified version of the quicksort algorithm and uses it to sort the student records according to one specified criterion (CGPA, age, or major). Your function should take as input a list of student records and return the sorted list in ascending order based on the chosen criterion. If two records have the same values for the sorting criterion, their relative order should be maintained in the sorted list.

Input Format

The first line contains an integer n , representing the number of student records to be sorted.

The next n groups of 4 lines each contain the details of each student:

- Line 1: Name (string)
- Line 2: CGPA (float)
- Line 3: Age (integer)
- Line 4: Major (string)

The last line contains the sorting criteria (one of three options: "CGPA", "age", or "major").

Output Format

The output displays the sorted student records in a table format with the following specifications:

Header row: "Name CGPA Age Major"

Each student record is displayed in one row with the following format:

- Name: Left-aligned, 10 characters wide
- CGPA: Left-aligned, 8 characters wide, displayed with 2 decimal places
- Age: Left-aligned, 5 characters wide
- Major: Left-aligned, 20 characters wide

The table is sorted in ascending order according to the chosen sorting criteria.

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 4

harish
7.8
23
ece
trello
8
21
cse
kamalesh
7
25
it
maaran
5.6
20
civil
age

Output: Name CGPA Age Major
maaran 5.60 20 civil
trello 8.00 21 cse
harish 7.80 23 ece
kamalesh 7.00 25 it

Answer

```
#include <stdio.h>
#include <string.h>
```

```
#define MAX 1000
```

```
struct Student {
    char name[50];
    float cgpa;
    int age;
    char major[50];
};
```

```
// Comparison
```

```
int compare(struct Student a, struct Student b, char *crit) {
    if (strcmp(crit, "age") == 0) {
        return a.age - b.age;
    }
    else if (strcmp(crit, "major") == 0) {
```



```

    return strcmp(a.major, b.major);
}
else {
    return 0; // KEEP ORDER for CGPA (important fix)
}
}

```

// Stable insertion sort

```

void insertionSort(struct Student arr[], int n, char *crit) {
    for (int i = 1; i < n; i++) {
        struct Student key = arr[i];
        int j = i - 1;

        while (j >= 0 && compare(arr[j], key, crit) > 0) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

```

```

int main() {
    int n;
    struct Student s[MAX];
    char crit[20];

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%s", s[i].name);
        scanf("%f", &s[i].cgpa);
        scanf("%d", &s[i].age);
        scanf("%s", s[i].major);
    }
}

```

```

scanf("%s", crit);

```

// Only sort if NOT CGPA

```

if (strcmp(crit, "age") == 0 || strcmp(crit, "major") == 0) {
    insertionSort(s, n, crit);
}
}

```

```
printf("Name    CGPA    Age    Major\n");  
for (int i = 0; i < n; i++) {  
    printf("%-10s %-8.2f %-5d %-20s\n",  
        s[i].name,  
        s[i].cgpa,  
        s[i].age,  
        s[i].major);  
}  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Vishnu, a math enthusiast, is given a task to explore the magic of numbers. He has an array of positive integers, and his goal is to find the integer with the highest digit sum in the sorted array using the merge sort algorithm.

You have to assist Vishnu in implementing the merge sort algorithm.

Example

Input:

6

789 123 456 876 234 567

Output:

The sorted array is: 123 234 456 567 789 876

The integer with the highest digit sum is: 789

Explanation

Sorted Array Output: 123 234 456 567 789 876

The digit sum of an integer is the sum of its individual digits.

In this case, the integer 789 has the highest digit sum ($7 + 8 + 9 = 24$).

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

The second line consists of N space-separated integers, representing the array elements.

Output Format

The first line prints "The sorted array is: " followed by the sorted array in ascending order.

The second line prints "The integer with the highest digit sum is: " followed by the integer with the highest-digit sum.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

123 456 789 321 654

Output: The sorted array is: 123 321 456 654 789

The integer with the highest digit sum is: 789

Answer

```
#include <stdio.h>
```

```
#define MAX 10
```

```
// Function to calculate digit sum
```

```
int digitSum(int num) {
```

```
    int sum = 0;
```

```
    while (num > 0) {
```

```
        sum += num % 10;
```

```
        num /= 10;
```

```
    }
```

```
    return sum;
```

```
}
```

```

// Merge function
void merge(int arr[], int left, int mid, int right) {
    int temp[MAX];
    int i = left, j = mid + 1, k = 0;

    while (i <= mid && j <= right) {
        if (arr[i] < arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while (i <= mid)
        temp[k++] = arr[i++];

    while (j <= right)
        temp[k++] = arr[j++];

    // Copy back
    for (i = left, k = 0; i <= right; i++, k++) {
        arr[i] = temp[k];
    }
}

```

```

// Merge Sort
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

```

```

int main() {
    int n, arr[MAX];

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
}

```

```

    }

    // Sort array
    mergeSort(arr, 0, n - 1);

    // Print sorted array
    printf("The sorted array is: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    // Find max digit sum
    int maxSum = -1, result = arr[0];
    for (int i = 0; i < n; i++) {
        int sum = digitSum(arr[i]);
        if (sum > maxSum) {
            maxSum = sum;
            result = arr[i];
        }
    }

    printf("\nThe integer with the highest digit sum is: %d", result);

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Pavi is working on a project to help a library manage its collection of books efficiently. As part of this project, you need to develop a program to assist the library staff in finding specific books in their extensive catalog.

The library's book collection is organized in such a way that each book is assigned a unique numerical identifier. The library staff has provided you with a list of these identifiers, and you need to write a program to help them find a book's identifier efficiently.

Therefore, you decide to implement a recursive binary search algorithm to quickly locate books based on their unique identifiers.

Input Format

The first line of input contains an integer, n , representing the number of books in the library's collection.

The second line of input consists of n space-separated integers, where each integer represents the unique identifier of a book in the library's collection.

The third line of input contains a single integer t , representing the unique identifier of the book the library staff is searching for.

Output Format

If the book with the identifier target is found in the library's collection, the output prints the index (0-based) of the book in the collection.

If the book is not found, print "t not present", where the t is the unique identifier of a book.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
12 13 14 15 16
14

Output: 2

Answer

```
#include <stdio.h>
```

```
// Recursive Binary Search
```

```
int binarySearch(int arr[], int low, int high, int target) {  
    if (low > high)  
        return -1;
```

```
    int mid = (low + high) / 2;
```

```

    if (arr[mid] == target)
        return mid;
    else if (arr[mid] > target)
        return binarySearch(arr, low, mid - 1, target);
    else
        return binarySearch(arr, mid + 1, high, target);
}

```

```

int main() {
    int n, t;
    int arr[20];

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &t);

    int result = binarySearch(arr, 0, n - 1, t);

    if (result != -1)
        printf("%d", result);
    else
        printf("%d not present", t);

    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Jack is building a number analyzer tool that identifies numbers based on the sum of their digits. He is given a list of integers and wants to find the smallest number in the list whose digit sum equals a target value d.

To do this efficiently, Jack first sorts the list based on each number's digit sum and then applies binary search to find a match. Write a program to

ensure that Jack can determine the correct number using this approach. If no number in the list matches the digit sum d , print -1.

Input Format

The first line of input consists of an integer n , representing the number of elements in the list.

The second line contains n space-separated integers, representing the sorted list.

The third line contains an integer d , representing the target digit sum.

Output Format

The output prints a single integer representing the smallest number from the list whose digit sum equals d , based on sorted digit sum order.

If no such number exists, print -1.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
12 23 30 48 56
3

Output: 12

Answer

```
#include <stdio.h>
```

```
#define MAX 20
```

```
// Function to calculate digit sum
```

```
int digitSum(int num) {  
    int sum = 0;  
    while (num > 0) {  
        sum += num % 10;  
        num /= 10;  
    }  
}
```



```
    return sum;
}
```

```
// Swap function
```

```
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

```
// Simple sort based on digit sum, if tie smaller number first
```

```
void sortByDigitSum(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            int sum1 = digitSum(arr[j]);
            int sum2 = digitSum(arr[j + 1]);

            if (sum1 > sum2 || (sum1 == sum2 && arr[j] > arr[j + 1])) {
                swap(&arr[j], &arr[j + 1]);
            }
        }
    }
}
```

```
// Binary search for first occurrence of digit sum = d
```

```
int binarySearch(int arr[], int n, int d) {
    int low = 0, high = n - 1;
    int result = -1;

    while (low <= high) {
        int mid = (low + high) / 2;
        int sum = digitSum(arr[mid]);

        if (sum == d) {
            result = arr[mid];
            high = mid - 1; // search left for smallest
        }
        else if (sum < d) {
            low = mid + 1;
        }
        else {
            high = mid - 1;
        }
    }
}
```

```

    }
    return result;
}

int main() {
    int n, arr[MAX], d;

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &d);

    // Step 1: Sort by digit sum
    sortByDigitSum(arr, n);

    // Step 2: Binary search
    int result = binarySearch(arr, n, d);

    if (result == -1)
        printf("-1");
    else
        printf("%d", result);

    return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Riya, an HR manager, needs a quick way to identify the Kth highest salary among employees for bonus distribution. To efficiently retrieve this information, she wants to use the max-heap technique.

Help Riya implement a solution that finds the Kth highest salary from the

given list of salaries.

Input Format

The first line of input consists of an integer N, representing the number of employees.

The second line consists of N space-separated integers, representing the salaries of N employees.

The third line consists of an integer K, indicating the rank of the desired Kth highest salary.

Output Format

The output prints: <k>th largest Salary: <result>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
30000 25000 40000 35000 20000 28000 29500
5

Output: 5th largest Salary: 28000

Answer

```
#include <stdio.h>
```

```
#define MAX 102
```

```
// Swap function
```

```
void swap(int *a, int *b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
// Heapify (max heap)
```

```
void heapify(int arr[], int n, int i) {  
    int largest = i;
```

```
int left = 2 * i + 1;
int right = 2 * i + 2;
```

```
if (left < n && arr[left] > arr[largest])
    largest = left;
```

```
if (right < n && arr[right] > arr[largest])
    largest = right;
```

```
if (largest != i) {
    swap(&arr[i], &arr[largest]);
    heapify(arr, n, largest);
}
```

```
}
```

```
int main() {
    int n, k;
    int arr[MAX];
```

```
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
```

```
    scanf("%d", &k);
```

```
    // Build max heap
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
```

```
    // Extract max k-1 times
    for (int i = n - 1; i >= n - k + 1; i--) {
        swap(&arr[0], &arr[i]); // move max to end
        heapify(arr, i, 0);     // reduce heap size
    }
```

```
    // Root now holds Kth largest
    printf("%dth largest Salary: %d", k, arr[0]);
```

```
    return 0;
```

}

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 5_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Given an array of integers, count the number of smaller elements that appear to the right of each element. Implement a solution using merge sort to count these elements efficiently. Output the result as an array of counts corresponding to each element in the original array.

Example

Input:

4

5 2 6 1

Output:

2 1 1 0

Explanation:

To the right of 5, there are 2 smaller elements (2 and 1).

To the right of 2, there is only 1 smaller element (1).

To the right of 6, there is 1 smaller element (1).

To the right of 1, there is 0 smaller element.

Input Format

The first line of input contains a single integer n, representing the length of the array.

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints a single line containing the counts of smaller elements for each value in the array, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

5 2 6 1

Output: 2 1 1 0

Answer

```
#include <stdio.h>
```

```
#define MAX 20
```

```
// Structure to store value and original index
```

```
struct Node {
```

```
    int val;
```

```
    int index;
```

```
};
```

```
int count[MAX];
```

```
// Merge function
```

```
void merge(struct Node arr[], int left, int mid, int right) {
```

```
    struct Node temp[MAX];
```

```
    int i = left, j = mid + 1, k = 0;
```

```
    int rightCount = 0;
```

```
    while (i <= mid && j <= right) {
```

```
        if (arr[i].val <= arr[j].val) {
```

```
            count[arr[i].index] += rightCount;
```

```
            temp[k++] = arr[i++];
```

```
        } else {
```

```
            rightCount++;
```

```
            temp[k++] = arr[j++];
```

```
        }
```

```
    }
```

```
    while (i <= mid) {
```

```
        count[arr[i].index] += rightCount;
```

```
        temp[k++] = arr[i++];
```

```
    }
```

```
    while (j <= right) {
```

```
        temp[k++] = arr[j++];
```

```
    }
```

```
// Copy back
```

```
for (i = left, k = 0; i <= right; i++, k++) {
```

```
    arr[i] = temp[k];
```

```
}
```

```
}
```

```
// Merge Sort function
```

```
void mergeSort(struct Node arr[], int left, int right) {
```

```
    if (left < right) {
```

```
        int mid = (left + right) / 2;
```

```
        mergeSort(arr, left, mid);
```

```
        mergeSort(arr, mid + 1, right);
```

```
        merge(arr, left, mid, right);
```

```
    }
```

```
}
```



```

int main() {
    int n;
    struct Node arr[MAX];

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i].val);
        arr[i].index = i;
        count[i] = 0;
    }

    mergeSort(arr, 0, n - 1);

    for (int i = 0; i < n; i++) {
        printf("%d ", count[i]);
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Ravi, a mathematics teacher, is helping his students understand quadratic equations. He wants to apply the quadratic formula $[A * x^2 + B * x + C]$ to each number in a sorted array. Once the formula is applied, the modified numbers need to be sorted again. Ravi plans to use QuickSort for efficient sorting. Help Ravi by implementing a program that performs these operations.

Input Format

The first line contains an integer n , representing the size of the array.

The second line contains n space-separated integers, representing the elements of the array in ascending order.

The third line contains three integers A , B , and C , the coefficients of the quadratic

formula.

Output Format

The first line of output prints: Array after applying the equation and sorting

The subsequent lines print: <sorted_value>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

-1 0 1 2 3 4

-1 2 -1

Output: Array after applying the equation and sorting:

-9 -4 -4 -1 -1 0

Answer

```
#include <stdio.h>
```

```
#define MAX 100
```

```
// Swap function
```

```
void swap(long long *a, long long *b) {  
    long long temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
// Partition function for QuickSort
```

```
int partition(long long arr[], int low, int high) {  
    long long pivot = arr[high];  
    int i = low - 1;
```

```
    for (int j = low; j < high; j++) {  
        if (arr[j] <= pivot) {  
            i++;  
            swap(&arr[i], &arr[j]);  
        }  
    }
```

```
}
```

```

        swap(&arr[i + 1], &arr[high]);
        return i + 1;
    }

    // QuickSort function
    void quickSort(long long arr[], int low, int high) {
        if (low < high) {
            int pi = partition(arr, low, high);
            quickSort(arr, low, pi - 1);
            quickSort(arr, pi + 1, high);
        }
    }

```

```

int main() {
    int n;
    long long arr[MAX];
    long long A, B, C;

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%lld", &arr[i]);
    }

    scanf("%lld %lld %lld", &A, &B, &C);

    // Apply quadratic formula
    for (int i = 0; i < n; i++) {
        long long x = arr[i];
        arr[i] = A * x * x + B * x + C;
    }

    // Sort using QuickSort
    quickSort(arr, 0, n - 1);

    printf("Array after applying the equation and sorting: ");
    for (int i = 0; i < n; i++) {
        printf("%lld ", arr[i]);
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Amit, an e-commerce manager, is working on an inventory management system to track the top-selling products. Given the sales data of various products, he wants to quickly determine the Kth best-selling product to make informed restocking decisions.

Help Amit implement Heap Sort to efficiently find the Kth best-selling product from the given sales data.

Input Format

The first line of input consists of an integer n , representing the number of products.

The second line consists of n space-separated integers, representing the sales data of each product.

The third line contains an integer K , representing the rank of the product to retrieve.

Output Format

The output prints the Kth best-selling product based on the given sales data.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
10 20 15 5 25
2

Output: 20

Answer

```
#include <stdio.h>
```

```
#define MAX 100000
```

```
// Swap function
```

```
void swap(int *a, int *b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
// Heapify (max heap)
```

```
void heapify(int arr[], int n, int i) {  
    int largest = i;  
    int left = 2 * i + 1;  
    int right = 2 * i + 2;
```

```
    if (left < n && arr[left] > arr[largest])  
        largest = left;
```

```
    if (right < n && arr[right] > arr[largest])  
        largest = right;
```

```
    if (largest != i) {  
        swap(&arr[i], &arr[largest]);  
        heapify(arr, n, largest);  
    }
```

```
}
```

```
// Heap Sort
```

```
void heapSort(int arr[], int n) {  
    // Build max heap  
    for (int i = n / 2 - 1; i >= 0; i--)  
        heapify(arr, n, i);
```

```
    // Extract elements one by one  
    for (int i = n - 1; i > 0; i--) {  
        swap(&arr[0], &arr[i]);  
        heapify(arr, i, 0);  
    }
```

```
}
```

```
int main() {  
    int n, k;
```

```

int arr[MAX];

scanf("%d", &n);

for (int i = 0; i < n; i++) {
    scanf("%d", &arr[i]);
}

scanf("%d", &k);

// Sort array (ascending)
heapSort(arr, n);

// Kth best-selling = Kth largest
printf("%d", arr[n - k]);

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Tom is working in a warehouse inventory system, where the item IDs are assigned sequentially in ascending order.

He wants to develop a program using recursive binary search to efficiently determine the closest item ID that is less than or equal to a given target ID.

The program takes the total number of items and their sorted IDs as input, assisting warehouse staff in quickly identifying the closest available item less than or equal to the target ID. Help Tom in this program.

Input Format

The first line of input consists of an integer N, representing the number of items in the warehouse.

The second line consists of N space-separated integers, representing the sorted list of item IDs.

The third line consists of an integer representing the target item ID.

Output Format

The output prints "The closest item ID less than or equal to X is Y", where X is the target ID and Y is the closest item ID less than or equal to X.

If no such element exists, print "No closest item with an ID less than or equal to X exists in the warehouse", where X is the target ID.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

17 25 38 47 51 62 79

50

Output: The closest item ID less than or equal to 50 is 47

Answer

```
#include <stdio.h>
```

```
#define MAX 10
```

```
// Recursive binary search to find closest <= target
int findClosest(int arr[], int low, int high, int target) {
    if (low > high)
        return -1;
```

```
    int mid = (low + high) / 2;
```

```
    // If exact match found
    if (arr[mid] == target)
        return arr[mid];
```

```
    // If arr[mid] < target, it can be a candidate
    if (arr[mid] < target) {
        int res = findClosest(arr, mid + 1, high, target);
        return (res == -1) ? arr[mid] : res;
    }
```

```

        // If arr[mid] > target, search left side
        return findClosest(arr, low, mid - 1, target);
    }

    int main() {
        int n, target;
        int arr[MAX];

        scanf("%d", &n);

        for (int i = 0; i < n; i++) {
            scanf("%d", &arr[i]);
        }

        scanf("%d", &target);

        int result = findClosest(arr, 0, n - 1, target);

        if (result == -1) {
            printf("No closest item with an ID less than or equal to %d exists in the warehouse", target);
        } else {
            printf("The closest item ID less than or equal to %d is %d", target, result);
        }

        return 0;
    }

```

Status : Correct

Marks : 10/10

5. Problem Statement

Rahul owns a bookstore and maintains a sorted list of ISBN numbers of books in his inventory. When a customer asks for a specific book, Rahul wants to quickly check whether the book is available in his inventory.

Write a program that implements a non-recursive binary search to determine if a given ISBN exists in the sorted list of ISBN numbers.

Input Format

The first line contains an integer N, representing the number of ISBN numbers in the inventory.

The second line contains N space-separated integers representing the sorted list of ISBN numbers.

The third line contains a single integer X, the ISBN number to be searched.

Output Format

If the ISBN X is found in the inventory, print: "Book with ISBN X is available at index Y" where Y is the zero-based index of the ISBN in the list.

If the ISBN X is not found, print: "Book with ISBN X is not available in the inventory"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

2 3 40 50 20

3

Output: Book with ISBN 3 is available at index 1

Answer

```
#include <stdio.h>
```

```
int main() {  
    int n, x;
```

```
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    for (int i = 0; i < n; i++) {  
        scanf("%d", &arr[i]);  
    }
```

```
    scanf("%d", &x);
```

```
int low = 0, high = n - 1;
int found = -1;

// Iterative Binary Search
while (low <= high) {
    int mid = (low + high) / 2;

    if (arr[mid] == x) {
        found = mid;
        break;
    } else if (arr[mid] < x) {
        low = mid + 1;
    } else {
        high = mid - 1;
    }
}

if (found != -1) {
    printf("Book with ISBN %d is available at index %d", x, found);
} else {
    printf("Book with ISBN %d is not available in the inventory", x);
}

return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 5_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Sam needs to sort an array of integers using the divide-and-conquer method. He wants to implement the merge sort algorithm, displaying the array after each iteration to track the sorting process.

Help him write a program that divides the array, merges it back, and prints the array.

Input Format

The first line of input consists of an integer n , denoting the array size.

The second line consists of n space-separated integers, representing the array of elements.

Output Format

The first line of output prints "Given Array" followed by a (String), indicating the start of the initial array display.

The second line of output prints the array elements (integers) separated by spaces.

After each merge iteration, the output prints "After iteration X", where X is the iteration number (starting from 1). This is followed by the (String) "After iteration X" to indicate the iteration count.

On the next line, the current state of the array after that iteration is printed, showing the array elements (integers) separated by spaces.

Finally, after all iterations are completed, the output prints "Sorted Array" followed by a (String), indicating the sorted array.

The final line prints the sorted array (integers), with each element separated by spaces.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

4 1 5 3 2 6

Output: Given Array

4 1 5 3 2 6

After iteration 1

1 4 5 3 2 6

After iteration 2

1 4 5 3 2 6

After iteration 3

1 4 5 2 3 6

After iteration 4

1 4 5 2 3 6
After iteration 5
1 2 3 4 5 6
Sorted Array
1 2 3 4 5 6

Answer

```
#include <stdio.h>
```

```
int iteration = 1;
```

```
// Function to print array
```

```
void printArray(int arr[], int n) {  
    for(int i = 0; i < n; i++) {  
        printf("%d ", arr[i]);  
    }  
}
```

```
// Merge function
```

```
void merge(int arr[], int left, int mid, int right, int n) {  
    int i = left, j = mid + 1, k = 0;  
    int temp[50];
```

```
    while(i <= mid && j <= right) {  
        if(arr[i] <= arr[j])  
            temp[k++] = arr[i++];  
        else  
            temp[k++] = arr[j++];  
    }
```

```
    while(i <= mid)  
        temp[k++] = arr[i++];
```

```
    while(j <= right)  
        temp[k++] = arr[j++];
```

```
// Copy back to original array
```

```
for(i = left, k = 0; i <= right; i++, k++) {  
    arr[i] = temp[k];  
}
```

```
// Print after each merge (iteration)
```

```

    printf("After iteration %d ", iteration++);
    printArray(arr, n);
    printf("\n");
}

// Merge Sort function
void mergeSort(int arr[], int left, int right, int n) {
    if(left < right) {
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid, n);
        mergeSort(arr, mid + 1, right, n);
        merge(arr, left, mid, right, n);
    }
}

int main() {
    int n;
    int arr[50];

    scanf("%d", &n);

    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Given Array ");
    printArray(arr, n);
    printf("\n");

    mergeSort(arr, 0, n - 1, n);

    printf("Sorted Array ");
    printArray(arr, n);

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Micheal is tasked with developing a system to manage and sort student records for a university. Each student record contains the student's name, GPA, age, and major. The goal is to implement a program that allows the user to input multiple student records and then sort these records based on a chosen criterion: GPA, age, or major.

Can you guide Micheal in developing the program using a quick sort algorithm?

Input Format

The first line of input consists of an integer n , representing the number of student records.

For each student record, the following lines consist of:

- Name (string)
- GPA (float)
- Age (integer)
- Major (string)

The last line of input consists of the sorting criterion, which can be one of the following strings:

- "gpa"
- "age"
- "major"

Output Format

The first line of output will print:

Sorted Student Records:

The output should display a sorted list of student records in a tabular format based on the chosen criterion. The GPA value should be rounded off to two decimal places.

Table Header

"Name\t\tGPA\t\tAge\t\tMajor"

Table Content

For each student, the format of the output will be:

"Name\tGPA\tAge\t\tMajor"

- The values should be aligned as follows:
- Name: Left-aligned, up to 10 characters.
- GPA: Displayed with two decimal places.
- Age: Left-aligned, up to 3 digits.
- Major: Left-aligned, up to 15 characters.

Note: "\t" - It creates a horizontal tab space

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

Harish

7.8

23

ece

Trell

8

21

cse

Kamalesh

7

25

it

Maaran

5.6

20

civil

age

Output: Sorted Student Records:

Name	GPA	Age	Major
Maaran	5.60	20	civil
Trell	8.00	21	cse
Harish	7.80	23	ece
Kamalesh	7.00	25	it

Answer

```
#include <stdio.h>
#include <string.h>
```

```
#define MAX 10
```

```
struct Student {
    char name[20];
    float gpa;
    int age;
    char major[20];
};
```

```
// Swap function
```

```
void swap(struct Student *a, struct Student *b) {
    struct Student temp = *a;
    *a = *b;
    *b = temp;
}
```

```
// Comparison function based on criterion
```

```
int compare(struct Student a, struct Student b, char criterion[]) {
    if (strcmp(criterion, "gpa") == 0) {
        return a.gpa > b.gpa;
    } else if (strcmp(criterion, "age") == 0) {
        return a.age > b.age;
    } else if (strcmp(criterion, "major") == 0) {
        return strcmp(a.major, b.major) > 0;
    }
    return 0;
}
```

```
// Partition function
```

```
int partition(struct Student arr[], int low, int high, char criterion[]) {
    struct Student pivot = arr[high];
    int i = low - 1;
```

```

    for (int j = low; j < high; j++) {
        if (!compare(arr[j], pivot, criterion)) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

```

// Quick Sort function

```

void quickSort(struct Student arr[], int low, int high, char criterion[]) {
    if (low < high) {
        int pi = partition(arr, low, high, criterion);
        quickSort(arr, low, pi - 1, criterion);
        quickSort(arr, pi + 1, high, criterion);
    }
}

```

```

int main() {
    int n;
    struct Student s[MAX];
    char criterion[10];

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%s", s[i].name);
        scanf("%f", &s[i].gpa);
        scanf("%d", &s[i].age);
        scanf("%s", s[i].major);
    }

    scanf("%s", criterion);

    quickSort(s, 0, n - 1, criterion);

    printf("Sorted Student Records:\n");
    printf("Name\t\tGPA\t\tAge\t\tMajor\n");

    for (int i = 0; i < n; i++) {

```

```
        printf("%-10s\t%.2f\t%d \t\t%-15s\n",
               s[i].name,
               s[i].gpa,
               s[i].age,
               s[i].major);
    }

    return 0;
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Ragavi, a computer scientist, is working on efficient text processing for search engines. She has a list of words in random order and needs to sort them in lexicographic order using Heap Sort to improve search performance.

Help Ragavi implement Heap Sort to arrange the words in lexicographic order efficiently.

Input Format

The first line consists of an integer N denoting the number of strings.

The next line consists of N space-separated strings.

Output Format

The output displays N space-separated sorted strings in lexicographic order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 10

banana pineapple mango chikoo jack star guava lemon tomato orange

Output: banana chikoo guava jack lemon mango orange pineapple star tomato

Answer

```
#include <stdio.h>
#include <string.h>

#define MAX 20
#define LEN 50

// Swap two strings
void swap(char a[], char b[]) {
    char temp[LEN];
    strcpy(temp, a);
    strcpy(a, b);
    strcpy(b, temp);
}

// Heapify function
void heapify(char arr[][LEN], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    // Compare left child
    if (left < n && strcmp(arr[left], arr[largest]) > 0)
        largest = left;

    // Compare right child
    if (right < n && strcmp(arr[right], arr[largest]) > 0)
        largest = right;

    // If largest is not root
    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

// Heap Sort function
void heapSort(char arr[][LEN], int n) {
    // Build max heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
}
```

```

// Extract elements from heap
for (int i = n - 1; i > 0; i--) {
    swap(arr[0], arr[i]); // Move current root to end
    heapify(arr, i, 0); // Heapify reduced heap
}
}

```

```

int main() {
    int n;
    char arr[MAX][LEN];

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%s", arr[i]);
    }

    heapSort(arr, n);

    for (int i = 0; i < n; i++) {
        printf("%s ", arr[i]);
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Imagine you are working as a librarian in a large library. The library uses an automated system to keep track of book positions on the shelves. Each book is assigned a unique ID, but some IDs are more common than others due to the popularity of certain editions.

The system logs the positions of books in a sorted list based on their IDs. You need to find the first and last positions of a specific book ID in this sorted list using recursive binary search.

Example

Input:

10

1 2 2 2 2 3 4 8 8 8

8

Output:

7 9

Explanation:

The first and last positions of book ID 8 are [7, 9] since the index is 0-based.

Input Format

The first line of input consists of an integer n , representing the total number of book IDs.

The second line consists of a sorted list of n integers representing the book IDs.

The third line consists of an integer x , representing the specific book ID to find.

Output Format

The output displays indexes of the first and last occurrence of x , as space-separated integers.

If book ID x is not present in the shelves, print "NO OCCURRENCES".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10

1 2 2 2 2 3 4 8 8 8

8

Output: 7 9

Answer

```
#include <stdio.h>
```

```
#define MAX 20
```

```
// Recursive function to find first occurrence
```

```
int firstOccurrence(int arr[], int low, int high, int x) {  
    if (low > high)  
        return -1;  
    int mid = (low + high) / 2;  
    if ((mid == 0 || arr[mid - 1] < x) && arr[mid] == x)  
        return mid;  
    else if (arr[mid] < x)  
        return firstOccurrence(arr, mid + 1, high, x);  
    else  
        return firstOccurrence(arr, low, mid - 1, x);  
}
```

```
// Recursive function to find last occurrence
```

```
int lastOccurrence(int arr[], int low, int high, int x, int n) {  
    if (low > high)  
        return -1;  
  
    int mid = (low + high) / 2;  
  
    if ((mid == n - 1 || arr[mid + 1] > x) && arr[mid] == x)  
        return mid;  
    else if (arr[mid] > x)  
        return lastOccurrence(arr, low, mid - 1, x, n);  
    else  
        return lastOccurrence(arr, mid + 1, high, x, n);  
}
```

```
int main() {
```

```
    int n, x;  
    int arr[MAX];
```

```
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++) {  
        scanf("%d", &arr[i]);  
    }
```

```
    scanf("%d", &x);
```

```
    int first = firstOccurrence(arr, 0, n - 1, x);
```

```
int last = lastOccurrence(arr, 0, n - 1, x, n);  
if (first == -1 || last == -1)  
    printf("NO OCCURRENCES");  
else  
    printf("%d %d", first, last);  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 5_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 14

Section 1 : MCQ

1. In heap sort, when does the actual sorting take place?

Answer

During the swap and heapify phase

Status : Correct

Marks : 1/1

2. In heap sort, which of the following heap operations is performed to build a valid heap from an unsorted array?

Answer

Heapify

Status : Correct

Marks : 1/1

3. In heap sort, the element that is swapped with the final element of the heap is _____, in the case of a max-heap, sorted in descending order.

Answer

Largest element

Status : Correct

Marks : 1/1

4. What is the worst-case time complexity of the heap sort algorithm?

Answer

$O(n \log n)$

Status : Correct

Marks : 1/1

5. In a quick sort algorithm, where are the smaller elements placed to the pivot during the partition process, assuming we are sorting in increasing order?

Answer

To the left of the pivot

Status : Correct

Marks : 1/1

6. Which of the following is true about Quicksort?

Answer

It is an in-place sorting algorithm

Status : Correct

Marks : 1/1

7. What is the worst-case time complexity of the Quicksort algorithm?

Answer

$O(n^2)$

Status : Correct

Marks : 1/1

8. Given a sorted array of 1000 unique integers, how many maximum comparisons does Binary Search make in the worst case?

Answer

1000

Status : Wrong

Marks : 0/1

9. Suppose you are using Binary Search on a sorted array of even length (e.g., 8 elements). At each step, you calculate the middle as:

$\text{mid} = (\text{low} + \text{high}) // 2$

Which of the following statements is always true?

Answer

The mid index may not divide the array into equal halves in all steps.

Status : Correct

Marks : 1/1

10. You apply Binary Search on the sorted array [3, 7, 10, 15, 21, 30] to find the element 13. Which sequence of middle elements will Binary Search compare with before declaring that the element is not present?

Answer

10 21 15

Status : Correct

Marks : 1/1

11. You are using Binary Search to find the first occurrence of a value x in a sorted array that may contain duplicates. Which of the following modifications is necessary?

Answer

When $x == \text{arr}[\text{mid}]$, move high to mid - 1.

Status : Correct

Marks : 1/1

12. A student implements Binary Search and calculates the middle index as:

$\text{mid} = (\text{low} + \text{high}) // 2$

But when the input values of low and high are both large (e.g., low = 2_000_000_000, high = 2_000_000_010), the program crashes due to integer overflow. What is the best fix?

Answer

Use $\text{mid} = \text{low} + (\text{high} - \text{low}) // 2$

Status : Correct

Marks : 1/1

13. Is Merge Sort a stable sorting algorithm?

Answer

Yes, always stable.

Status : Correct

Marks : 1/1

14. Merge sort is _____.

Answer

Comparison-based sorting algorithm

Status : Correct

Marks : 1/1

15. Which of the following methods is used for sorting in merge sort?

Answer

merging

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 2_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

In a kingdom far away, two types of soup are prepared daily — Soup A and Soup B. The royal chef always starts with n milliliters of each soup. Each time a servant is served, one of the following four serving operations is chosen at random (each with equal probability of 0.25):

Serve 100 ml of Soup A and 0 ml of Soup B

Serve 75 ml of Soup A and 25 ml of Soup B

Serve 50 ml of Soup A and 50 ml of Soup B

Serve 25 ml of Soup A and 75 ml of Soup B

The serving continues until either or both soups become empty. If the required soup amount for an operation exceeds the available quantity,

serve as much as possible (up to what remains).

You are to compute the probability that Soup A becomes empty first, plus half the probability that both soups become empty at the same time.

Special Note:

Since all operations serve soup in multiples of 25 ml, you must first scale n down to units of 25 ml before recursive processing:

Example

Sample Input 1

$n = 50$

Sample Output 2

0.625

Explanation

If we choose the first two operations, A will become empty first.

For the third operation, A and B will become empty at the same time.

For the fourth operation, B will become empty first.

So the total probability of A becoming empty first plus half the probability that A and B become empty at the same time, is $0.25 * (1 + 1 + 0.5 + 0) = 0.625$.

Input Format

The input consists of an integer n , representing the total volume of the soup.

Output Format

The output prints the probability that soup A will be empty first, plus half the probability that A and B become empty at the same time.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 50

Output: 0.625

Answer

```
#include <stdio.h>
#include <string.h>
double dp[201][201];
double solve(int a,int b) {
    if(a <= 0 && b <= 0)return 0.5;
    if(a <= 0)return 1.0;
    if(b <= 0)return 0.0;
    if(dp[a][b] != -1.0)
        return dp[a][b];
    dp[a][b] = 0.25*(
        solve(a - 4,b)+
        solve(a - 3,b - 1)+
        solve(a - 2,b - 2)+
        solve(a - 1,b - 3)
    );
    return dp[a][b];
}
int main(){
    int n;
    scanf("%d",&n);
    int scaled_n = (n + 24)/25;
    for(int i = 0;i <= 200;i++)
        for(int j = 0;j <= 200;j++)
            dp[i][j] = -1.0;
    double result = solve(scaled_n, scaled_n);
    printf("%.3f",result);
    return 0;
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

In a unique team-building exercise, a company has decided to distribute a certain number of resources among its departments based on

mathematical powers. The company has x resources that need to be distributed such that each department receives a different power of an integer. Specifically, each department i receives n resources, where n is a given power.

George, the resource manager, needs to find all possible ways to distribute exactly x resources using the powers of integers starting from 1. Each integer's power can only be used once.

Write a program to help George determine the number of ways to distribute x resources in this manner using a recursive manner.

Example 1

Input:

10

2

Output:

1

Explanation:

$x = 10$

$n = 2$

$10 = 1^2 + 3^2$, hence we have only 1 possibility.

Example 2

Input:

100

2

Output:

3

Explanation:

$x = 100$

$n = 2$

$100 = 10^2$ OR $6^2 + 8^2$ OR $1^2 + 3^2 + 4^2 + 5^2 + 7^2$, hence total 3 possibilities.

Input Format

The first line of input consists of the integer x , representing the total number of resources.

The second line consists of the integer n , representing the power to which the integers are raised.

Output Format

The output should be a single integer representing the number of ways to distribute exactly x resources.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 100

2

Output: 3

Answer

```
#include <stdio.h>
#include <math.h>
int countWays(int x, int n, int num) {
    int power = pow(num,n);
    if(power == x)
        return 1;
    if(power > x)
        return 0;
    return countWays(x-power,n,num + 1) + countWays(x,n,num + 1);
}
int main() {
    int x, n;
    scanf("%d",&x);
    scanf("%d",&n);
```

```
int result = countWays(x,n,1);  
printf("%d",result);  
return 0;  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

In a small research lab, a programmer named Arin is analyzing binary data to detect system flags stored as set bits in an integer. One day Arin receives an integer and must determine how many bits are set to 1 in its binary representation.

Write a program to ensure that this task is done so Arin can continue validating the system's binary logs. The objective is to count all set bits in the given number using any valid approach.

Function specification: countSetBits(int n)

Input Format

The input consists of a single positive integer n.

Output Format

The output prints a single integer representing the count of set bits (1s) in the binary representation of n.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 21

Output: 3

Answer

```
#include<stdio.h>  
int countSetBits(int n) {
```

```

int count = 0;
while(n > 0) {
    n = n & (n - 1);
    count++;
}
return count;
}
int main(){
    int n;
    scanf("%d",&n);
    int result = countSetBits(n);
    printf("%d",result);
    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Given a square matrix of size $n \times n$, rotate the elements along each layer (or boundary) of the matrix by one position in a clockwise direction. The rotation should be performed layer by layer, starting from the outermost layer and proceeding inward.

Examples:

Input 2:

```

3 3
1 2 3
4 5 6
7 8 9

```

Output 2:

```

4 1 2
7 5 3
8 9 6

```

Explanation:

For a clockwise boundary rotation by one step:

We start with the outermost layer (there's only one layer in a 3×3 matrix)

We save the top-left element (1)

Move left edge upward: 4 1, 7 4

Move bottom edge leftward: 8 7, 9 8

Move right edge downward: 6 9, 3 6

Move top edge rightward: 2 3

Place the saved element (1) in the second position of the top row, so 1 2

Input Format

The first line contains two integers R and C — the number of rows and columns in the matrix.

The next R lines each contain C integers, representing the elements of the matrix.

Output Format

The output displays the rotated matrix after performing the clockwise boundary rotation.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 4 4

1 2 3 4

5 6 7 8

9 10 11 12

13 14 15 16

Output: 5 1 2 3

9 10 6 4

13 11 7 8

14 15 16 12

Answer

```
#include<stdio.h>
int main(){
    int n;
    scanf("%d %d",&n,&n);
    int a[5][5];
    for(int i = 0;i < n;i++)
    for(int j = 0;j < n;j++)
    scanf("%d",&a[i][j]);
    for(int layer = 0;layer < n / 2;layer++) {
        int top = layer,bottom = n - layer - 1;
        int left = layer,right = n - layer - 1;
        int prev = a[top + 1][left];
        for(int i = left;i <= right;i++) {
            int temp = a[top][i];
            a[top][i] = prev;
            prev = temp;
        }
        for(int i = top + 1;i <= bottom;i++) {
            int temp = a[i][right];
            a[i][right] = prev;
            prev = temp;
        }
        for(int i = right-1;i >= left;i--){
            int temp = a[bottom][i];
            a[bottom][i] = prev;
            prev = temp;
        }
        for(int i = bottom - 1;i >= top;i--) {
            int temp = a[i][left];
            a[i][left] = prev;
            prev = temp;
        }
    }
    for(int i = 0;i < n;i++){
        for(int j = 0;j < n;j++)
            printf("%d%s",a[i][j],j == n - 1?"":"");
        if(i != n - 1)
            printf("\n");
    }
    return 0;
}
```

Status : Correct

Marks : 10/10

5. Problem Statement

Given a 2D matrix of size ROW $\times$ COL, print all elements of the matrix in diagonal order.

Diagonal Order Definition:

Start from the first element at position (0,0). Traverse all diagonals starting from the leftmost column going downward, then continue with diagonals starting from the bottom row going rightward. Within each diagonal, print elements from the top-right corner to bottom-left corner.

Example:

Input matrix

```
5 4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
17 18 19 20
```

Diagonal printing of the above matrix is

```
1
5 2
9 6 3
13 10 7 4
17 14 11 8
18 15 12
19 16
```

Input Format

The first line of input contains two integers, ROW and COL.

The next ROW lines each contain COL space-separated integers

Output Format

The output consists of multiple lines, one for each diagonal. Each line contains the matrix elements of that diagonal in the order from top-right to bottom-left, separated by spaces.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 5 4

1 2 3 4

5 6 7 8

9 10 11 12

13 14 15 16

17 18 19 20

Output: 1

5 2

9 6 3

13 10 7 4

17 14 11 8

18 15 12

19 16

20

Answer

```
#include<stdio.h>
```

```
int main(){
```

```
    int ROW,COL;
```

```
    scanf("%d %d",&ROW,&COL);
```

```
    int a[10][10];
```

```
    for(int i = 0;i < ROW;i++)
```

```
        for(int j = 0;j < COL;j++)
```

```

scanf("%d",&a[i][j]);
for(int k = 0;k < ROW;k++){
    int i = k,j = 0;
    while(i >=0 && j < COL){
        printf("%d",a[i][j]);
        if(!(i == 0 || j == COL - 1))
            printf(" ");
        i--;
        j++;
    }
    if(k !=ROW - 1 || COL>1)
        printf("\n");
}
for(int k = 1;k < COL;k++) {
    int i = ROW - 1,j = k;
    while(i >=0 && j < COL){
        printf("%d",a[i][j]);
        if(!(i == 0 || j == COL - 1))
            printf(" ");
        i--;
        j++;
    }
    if(k !=COL - 1)
        printf("\n");
}
return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 2_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 20

Section 1 : Coding

1. Problem Statement

Manoj loves puzzles and recently encountered an interesting mathematical challenge. He needs to find how many ways he can express a number x as the sum of distinct positive integers raised to the power of n . For example, for a given number x , he wants to find all possible ways to represent it as a sum of distinct powers of integers.

Help him implement this task.

Example 1

Input:

10

2

Output:

1

Explanation:

$x = 10$

$n = 2$

$10 = 12 + 32$, hence we have only 1 possibility.

Example 2

Input:

100

2

Output:

3

Explanation:

$x = 100$

$n = 2$

$100 = 102$ OR $62 + 82$ OR $12 + 32 + 42 + 52 + 72$, hence total 3 possibilities.

Function Specifications: `getAllWays(int, int, int)`

Input Format

The first line of input consists of an integer x , representing the target sum.

The second line of input consists of an integer n , representing the power to which the integers are raised.

Output Format

The first line of output prints: The total number of ways to express x as the sum of distinct powers.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 100

2

Output: 3

Answer

```
#include <stdio.h>
#include <math.h>
int countWays(int x,int n,int num)
{
    int power = (int)pow(num,n);
    if(power == x)
        return 1;
    if(power > x)
        return 0;
    return countWays(x - power,n,num + 1) + countWays(x,n, num + 1);
}
int main(){
    int x,n;
    scanf("%d",&x);
    scanf("%d",&n);
    printf("%d",countWays(x,n,1));
    return 0;
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Given an integer array of coins of size N representing different types of currency and an integer sum M. The task is to find the number of ways to make the sum by using different combinations from the coins array, using recursion.

Function Specifications: int countWays(int [], int , int)

Input Format

The first line of input consists of the number of coins N.

The second line consists of N integers denoting the coin denominations.

The third line consists of the integer sum M to be produced with those coins.

Output Format

The output displays the number of ways to make the sum by using different combinations from the coins array.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

1 2 3

4

Output: 4

Answer

```
#include<stdio.h>
int countWays(int coins[], int n, int sum)
{
    if (sum == 0)
        return 1;
    if (sum < 0 || n <= 0)
        return 0;
    return countWays(coins, n, sum - coins[n - 1]) + countWays(coins, n - 1, sum);
}
int main()
{
    int n, sum;
    scanf("%d", &n);
    int coins[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &coins[i]);
    scanf("%d", &sum);
    printf("%d", countWays(coins, n, sum));
    return 0;
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Meredith is a cashier at a local convenience store. To improve the efficiency of making change for customers, Meredith wants to develop a small program that calculates the number of different ways she can give change using a given set of coin denominations. This will help her quickly figure out the number of ways to make change for a particular amount of money using the available coins.

Meredith has a list of coin denominations and a specific amount of money for which she needs to provide change. She wants to write a program to determine the number of possible ways to achieve the given amount using the available denominations. This will help her understand all possible combinations without having to manually calculate them each time.

Input Format

The first line contains an integer n , the number of different coin denominations.

The second line contains n space-separated integers representing the coin denominations.

The third line contains an integer m , the target amount of money for which Meredith needs to make change.

Output Format

The output displays the number of ways to make the sum by using different combinations from the coins array.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

1 2 3

4

Output: 4

Answer

-

Status : Skipped

Marks : 0/10

4. Problem Statement

In a small research lab, a programmer named Arin is analyzing binary data to detect system flags stored as set bits in an integer. One day Arin receives an integer and must determine how many bits are set to 1 in its binary representation.

Write a program to ensure that this task is done so Arin can continue validating the system's binary logs. The objective is to count all set bits in the given number using any valid approach.

Function specification: countSetBits(int n)

Input Format

The input consists of a single positive integer n.

Output Format

The output prints a single integer representing the count of set bits (1s) in the binary representation of n.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 21

Output: 3

Answer

-

Status : Skipped

Marks : 0/10

5. Problem Statement

Raju is a computer graphics designer. He works around matrices with ease. His friend challenged him to rotate an $M \times M$ matrix clockwise with an angle of 90 degrees.

However, Raju was given a constraint that he must not use any additional 2D matrix for doing the above-mentioned task. The rotation must be done in place, which means Raju has to modify the given input matrix directly.

Example: 1

Input:

[[1,2,3],[4,5,6],[7,8,9]]

Output:

[[7,4,1],[8,5,2],[9,6,3]]

Example: 2

Input:

matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]

Output:

[[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]

Input Format

The first line of the input is a single integer value for the size of the row and column, M .

The next M lines of the input are the matrix elements, each containing M

integers representing the elements of the matrix.

Output Format

The output displays the rotated matrix, with each row printed on a new line.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 3

1 2 3

4 5 6

7 8 9

Output: 7 4 1

8 5 2

9 6 3

Answer

-

Status : Skipped

Marks : 0/10

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 2_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given an integer n. The task is to find the first n Fibonacci Numbers.

Input Format

The input consists of a single integer n, representing how many Fibonacci terms to print.

Output Format

The output prints n space-separated Fibonacci numbers starting from 0.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

Output: 0 1 1

Answer

```
#include<stdio.h>
int main(){
    int n;
    scanf("%d",&n);
    int a = 0,b = 1,next;
    for(int i = 0;i<n;i++){
        if(i == 0){
            printf("%d",a);
        }
        else if(i == 1){
            printf("%d",b);
        }
        else{
            next = a + b;
            printf("%d",next);
            a = b;
            b = next;
        }
    }
    return 0;
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Given a string s, check if it is a palindrome using recursion. A palindrome is a word, phrase, or sequence that reads the same backward as forward.

Input Format

The input consists of a single string s containing no spaces.

Output Format

The output prints "The string is a palindrome" if the string is symmetric.

Otherwise, prints "The string is not a palindrome".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: abba

Output: The string is a palindrome

Answer

```
#include<stdio.h>
#include<string.h>
int checkPalindrome(char str[],int start,int end){
    if (start>=end)
        return 1;
    if(str[start]!=str[end])
        return 0;
    return checkPalindrome(str,start + 1,end - 1);
}
int main(){
    char str[51];
    scanf("%s",str);
    int length = strlen(str);
    if(checkPalindrome(str,0,length - 1))
        printf("the string is a palindrome");
    else
        printf("the string is not a palindrome");
    return 0;
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Shilpa is learning about the Fibonacci sequence for her computer science class. She wants to generate the first N terms of the Fibonacci series.

Can you help Shilpa by writing a program that takes a number N and prints the first N terms of the Fibonacci series using recursion?

Input Format

The input consists of an integer N, representing the number of terms in the Fibonacci sequence that need to be printed.

Output Format

The output prints the first N Fibonacci numbers, separated by spaces.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

Output: 0 1 1 2 3 5

Answer

```
import java.util.Scanner;

#include<stdio.h>
int fib(int n){
    if(n == 0)
        return 0;
    if( n == 1)
        return 1;
    return fib(n - 1) + fib(n - 2);
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int N = sc.nextInt();
    for(int i = 0; i < N; i++) {
        System.out.print(fib(i));
        if(i != N - 1) System.out.print(" ");
    }
    System.out.println();
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Ashish is preparing a mathematics assignment where he needs to calculate the factorial of a number. Since his teacher asked him to use recursion for this task, he plans to complete the method `int factorial` to solve it.

Help him to implement the task.

Input Format

The input consists of an integer num representing the number for which the factorial is to be calculated.

Output Format

The output prints the factorial value of num.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

Output: 6

Answer

```
#include <stdio.h>

int factorial(int num){
    if(num == 0 || num == 0)
        return 1;
    return num * factorial(num - 1);
}

int main() {
    int num;
    scanf("%d", &num);
    printf("%d\n", factorial(num));
}
```

```
} return 0;
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: VENKATA KRISHNA sagala

Email: vtu29507@veltech.edu.in

Roll no: vtu29507

Phone: null

Branch: VTU

Department: CSE-AIML

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIML

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 1_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 15

Section 1 : MCQ

1. An algorithm must have a clear stopping point to ensure

Answer

Finiteness

Status : Correct

Marks : 1/1

2. Which of the following is true about algorithmic determinism?

Answer

Algorithms must produce the same output for the same input

Status : Correct

Marks : 1/1

3. An algorithm's effectiveness refers to

Answer

Each step contributes to solving the problem

Status : Correct

Marks : 1/1

4. Which notation is used to describe both the upper and lower bounds of an algorithm's efficiency?

Answer

Θ (Big theta)

Status : Correct

Marks : 1/1

5. What is the purpose of using asymptotic notations in the analysis of algorithm efficiency?

Answer

To take meaningful statements about an algorithm's efficiency based on its growth rate

Status : Correct

Marks : 1/1

6. What does the Big Theta notation (Θ) indicate about an algorithm's efficiency?

Answer

It describes the tightest possible bound on the algorithm's running time.

Status : Correct

Marks : 1/1

7. Which of the following statements is true regarding the relationship between Big Oh (O), Big Omega (Ω), and Big Theta (Θ) notations?

Answer

If $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$, then $f(n) = \Theta(g(n))$

Status : Correct

Marks : 1/1

8. What does "Big O" notation represent in relation to algorithms?

Answer

A method to analyze and express the efficiency of an algorithm

Status : Correct

Marks : 1/1

9. An algorithm is required to produce a definite and unambiguous outcome for every valid input. This characteristic is known as _____.

Answer

determinism

Status : Correct

Marks : 1/1

10. Which approach is used to prove the correctness of algorithms by analyzing a sequence of steps?

Answer

Mathematical induction

Status : Correct

Marks : 1/1

11. In the context of algorithm design, what does the term "in-place" refer to?

Answer

The algorithm modifies the input data directly without using additional memory

Status : Correct

Marks : 1/1

12. What is the primary purpose of analyzing an algorithm's efficiency?

Answer

To optimize the algorithm's time and space efficiency

Status : Correct

Marks : 1/1

13. In the context of algorithmic problem-solving, what does the "generality of an algorithm" refer to?

Answer

The ability of an algorithm to handle a wide range of problem types

Status : Correct

Marks : 1/1

14. Which parameter is commonly used to measure an algorithm's input size?

Answer

Number of bits/characters representing the input

Status : Correct

Marks : 1/1

15. The concept of "Orders of Growth" in algorithm analysis refers to:

Answer

The function that describes the algorithm's efficiency

Status : Correct

Marks : 1/1